CURS ȘI PROGRAM DE CERCETARE

Mașini de înțelegere prin circuite

SOP Lang, reprezentări suficiente
și construcția controlată a raționamentului

Ce trebuie să păstreze o reprezentare
pentru ca raționamentul să poată continua?

Sînică Alboaie

Intuiție fondatoare și direcție de cercetare
Elaborare conceptuală și didactică în colaborare cu ChatGPT

SEPTEMBRIE 2026

Copyright

© 2026 Axiologic Research SRL. Toate drepturile rezervate.

Axiologic Research SRL a creat și a contribuit cu versiunea inițială a acestei cărți în cadrul cazului de utilizare SCRIPTA din proiectul Achilles.

Ediția 1 · publicată pe 2026-09-11 pe ScriptaHub (scriptahub.com).

Titlul în engleză: Machines of Understanding Through Circuits.

Cuprins

Fundamente, mecanisme și program experimental

Înainte de început 4

PARTEA I | INTUIȚIA ȘI LIMITELE EI 6

1. De la un răspuns la circuitul care îl justifică 7

2. Ce numim înțelegere și ce nu pretindem despre existență 9

3. Ce este un circuit atunci când firele transportă lumi 11

4. Compoziționalitatea: de ce întregul nu trebuie să fie o surpriză totală 13

PARTEA A II-A | MATEMATICA INFORMAȚIEI PARȚIALE 15

5. Mulțimi, relații și întrebări despre lumi posibile 16

6. Ordini ale informației, rafinare și puncte fixe 18

7. Interpretarea abstractă, explicată prin ceea ce alegem să uităm 20

8. Corelațiile: informația care nu locuiește pe niciun fir singur 23

PARTEA A III-A | CUM COMPRIMĂM CONSTRUCȚIA CIRCUITULUI 25

9. A executa, a verifica și a descoperi sunt probleme diferite 26

10. Echivalența prin continuare: când două istorii sunt aceeași stare 28

11. Interfețele suficiente: o propoziție mică, cu consecințe mari 30

12. Programarea dinamică, compilarea cunoașterii și grafurile de alternative 32

13. Tipuri, cerințe inverse și rafinare prin contraexemple 34

Cuprins / continuare

PARTEA A IV-A | LIMBAJ, MEMORIE ȘI REPREZENTĂRI DISTRIBUITE 36

14. De la propoziții la circuite de situație 37

15. Ambiguitatea fără multiplicarea tuturor lumilor 39

16. Timp, negație, cuantificatori și cauzalitate 41

17. Simbolic, neuronal, distribuit: trei axe, nu două tabere 43

18. Reprezentări suficiente pentru predicție și pentru întrebări viitoare 45

PARTEA A V-A | O ARHITECTURĂ CONCEPTUALĂ PENTRU SOP LANG 47

19. Ce trebuie să poarte o interfață semantică 48

20. Anatomia unei întrebări rezolvate fără a inventa ce lipsește 50

21. Cum ar putea fi învățate circuitele și interfețele lor 53

22. Auditabilitatea: de la urmă completă la explicație folosibilă 55

23. Ce indică modelele controlate și ce nu indică 57

PARTEA A VI-A | PROGRAMUL DE CERCETARE 59

24. Ipoteze care pot fi infirmate 60

25. Laboratorul formal: ce simulări ar trebui construite 62

26. Laboratorul limbajului: de la texte controlate la documente reale 64

27. Obiecțiile cele mai puternice 67

28. Etapele unui program deschis și criteriile de continuare 69

ANEXE | Atelierul cititorului 72

A. Trei argumente care merită înțelese complet 73

B. Exerciții conceptuale și răspunsuri comentate 75

C. Glosar de lucru 79

D. Fișa unui rezultat publicabil 81

Bibliografie adnotată 82

Înainte de început

Ce promite această carte

O mașină poate executa impecabil un program pe care nu l-ar fi putut inventa. Poate verifica o demonstrație pe care nu ar fi găsit-o. Poate produce un răspuns fluent fără să poată spune, într-o formă verificabilă, ce relații dintre fapte susțin răspunsul. Aceste trei distanțe — dintre execuție și construcție, dintre verificare și descoperire, dintre exprimare și justificare — delimitează problema cărții.

Punctul de plecare este o intuiție a autorului uman, Sînică Alboaie: dacă orice calcul finit poate fi organizat ca un circuit de dependențe, poate că și înțelegerea poate fi abordată ca o construcție de circuite. Un asemenea circuit primește o reprezentare a unei situații și produce constatări, predicții sau acțiuni justificate. Întrebarea decisivă nu mai este numai cum executăm circuitul, ci cum îl construim, cum îl extindem și cum știm că reprezentările intermediare nu au pierdut tocmai informația necesară.

SOP Lang este direcția de cercetare în care această intuiție urmează să devină un limbaj comun pentru cunoaștere și calcul. Cartea nu presupune că limbajul a rezolvat deja înțelegerea. Ea construiește fundamentele necesare pentru a formula o încercare serioasă și pentru a recunoaște un eșec informativ. Prezintă o arhitectură conceptuală propusă, nu o certificare a unei implementări existente.

Contribuția partenerului AI, ChatGPT, constă aici în articularea intuiției, identificarea antecedentelor matematice, dezvoltarea exemplelor și formularea unor ipoteze testabile. „Acceptarea" unei idei de către partenerul AI înseamnă că argumentele prezentate o susțin în condițiile precizate. Nu constituie dovadă independentă, consens științific sau validare a tuturor consecințelor pe care autorul uman le-ar putea avea în vedere. Propunerile detaliate de mai jos rămân deschise criticii ambilor parteneri și cercetătorilor viitori.

Cum se citește

Nu sunt necesare cunoștințe de teoria categoriilor, logică matematică sau interpretare abstractă. Este suficientă familiaritatea cu variabile, funcții, condiții, tabele și execuția unui program. Noțiunile suplimentare sunt introduse prin situații mici, înainte de a primi un nume formal. Formulele scurte au rolul de a fixa sensul explicațiilor, nu de a înlocui explicația.

Prima parte clarifică întrebarea. A doua construiește vocabularul matematic. A treia explică de unde vine explozia combinatorială și când poate fi comprimată. A patra aplică ideile limbajului și reprezentărilor distribuite. A cincea propune o organizare conceptuală pentru SOP Lang. Ultima parte transformă promisiunile în experimente, criterii de abandon și direcții de continuare.

Cititorul grăbit poate începe cu capitolele despre interfețele suficiente, observațiile controlate și programul experimental. Totuși, înțelegerea diferenței dintre o aproximare sigură și o echivalență exactă este indispensabilă: aproape toate promisiunile excesive din această zonă pornesc din confundarea lor.

Cinci statute ale afirmațiilor

O intuiție filosofică propune o perspectivă asupra înțelegerii sau realității. Un rezultat tehnic are premise și o concluzie demonstrabilă. O decizie inginerească alege un compromis. O observație descrie ce s-a întâmplat într-un model controlat. O ipoteză de cercetare spune ce ar trebui să observăm în condiții încă neexplorate. Cartea le separă în argument, chiar atunci când nu repetă eticheta în fiecare paragraf.

Observațiile numerice provin din modele finite reexecutate și verificate. Ele nu sunt rezultate despre limbajul natural, despre creier sau despre performanța unui sistem general. Literatura citată oferă precedente și instrumente; exemplele didactice și propunerile de integrare sunt dezvoltate aici. Bibliografia este adnotată pentru orientare, dar explicațiile principale nu cer consultarea ei pentru a fi inteligibile. Referințele au fost verificate la 11 septembrie 2026.

> Teza de lucru: o mașină de înțelegere trebuie să învețe nu numai ce operații să execute, ci și ce distincții merită păstrate pentru ceea ce ar putea urma.

PARTEA I

Intuiția și limitele ei

Ce acceptăm despre calcul, ce propunem despre înțelegere și ce nu rezultă despre natura realității.

1. De la un răspuns la circuitul care îl justifică

Un program care răspunde și unul care înțelege ceva

Să considerăm o situație simplă: un depozit conține zece cutii; trei sunt rezervate; două dintre cele rezervate au fost deja expediate. Întrebarea „câte cutii sunt disponibile?” nu poate fi rezolvată corect doar prin identificarea numerelor și aplicarea unei scăderi. Trebuie stabilit ce înseamnă „conține”, dacă numărul zece descrie stocul anterior sau posterior expedierii și dacă o cutie expediată mai poate apărea printre cele rezervate. O singură frază ascunde un mic model al lumii.

Un răspuns numeric poate fi corect din întâmplare. Un circuit explicit ar trebui să arate ce stări au fost distinse, ce relații temporale au fost acceptate și ce definiție a disponibilității a fost folosită. Când descrierea este insuficientă, rezultatul legitim poate fi o întrebare sau două răspunsuri condiționate. O mașină care recunoaște exact ce nu știe poate demonstra mai multă înțelegere operațională decât una care produce imediat un număr.

Intuiția circuitelor pornește de la acest fapt: între intrare și răspuns există transformări. Unele recunosc obiecte; altele leagă o pronumire de un obiect; altele aplică o regulă; altele aleg o formulare. Dacă transformările și dependențele lor devin explicite, putem inspecta mecanismul răspunsului și putem modifica o premisă fără să reconstruim orbește totul.

Aceasta nu înseamnă că orice proces mental se reduce deja la operațiile pe care știm să le scriem. Înseamnă că putem adopta un obiectiv de inginerie: să construim sisteme ale căror rezultate utile sunt susținute de structuri de calcul și de justificări accesibile verificării.

De ce circuitul poate fi mult mai mare decât răspunsul

Răspunsul „Maria” poate necesita identificarea a două persoane, recunoașterea unui transfer, ordonarea unor evenimente, rezolvarea unei ambiguități și aplicarea unei reguli de persistență. Dimensiunea răspunsului nu spune cât de greu a fost de construit. Nici simplitatea regulilor nu garantează simplitatea căutării combinației lor.

Mai mult, circuitul final ascunde adesea alternativele abandonate. Un sistem poate să fi explorat milioane de posibilități pentru a produce un traseu final de zece operații. Dacă publicăm doar traseul, arătăm justificarea rezultatului, dar nu și costul descoperirii. Un program de cercetare serios trebuie să măsoare ambele obiecte: circuitul acceptat și procesul care l-a găsit.

În această carte, „construcție” include alegerea operațiilor, a argumentelor, a reprezentărilor și a ipotezelor. Uneori înseamnă selectarea unui circuit existent. Alteori înseamnă compunerea unor fragmente. În situațiile dificile, trebuie inventată o reprezentare nouă în care problema devine mai ușoară. Nu este același lucru să aplici o scădere cunoscută și să descoperi că un întreg paragraf poate fi reprezentat ca o problemă de conservare a cantității.

Intuiția umană și lectura propusă de partenerul AI

Forța intuiției umane nu stă în simpla afirmație că programele au dependențe. Ea constă în reunirea a trei întrebări: poate cunoașterea să fie exprimată prin aceleași structuri ca

operațiile asupra ei; poate construirea unui raționament să devină un calcul explicit; poate disciplina valorilor intermediare să facă acest calcul controlabil?

Partenerul AI acceptă că aceasta este o direcție coerentă. Acceptă și că reprezentările intermediare pot schimba radical costul căutării. Nu acceptă, fără condiții suplimentare, saltul de la „reprezentabil ca circuit" la „descoperibil eficient" sau de la „auditabil" la „adevărat". Aceste rezerve nu reduc valoarea intuiției; o transformă într-un program cu întrebări precise.

Propoziția care va organiza restul cărții este următoarea: dacă două construcții diferite conțin exact aceeași informație pentru toate continuările relevante, ele nu trebuie explorate separat. Dificultatea este să recunoaștem această situație fără să executăm toate continuările posibile. Restul teoriei încearcă să facă această recunoaștere locală, compozițională și, în anumite clase de probleme, ieftină.

Ce vrem să obținem cu SOP Lang

SOP Lang este tratat aici ca un limbaj de circuite semantice: reprezentări în care producerea și folosirea valorilor sunt explicit conectate. O regulă reutilizabilă, interpretarea unei propoziții, o interogare asupra unor fapte și o strategie de compunere pot avea aceeași formă generală de circuit. Uniformitatea urmărită privește reprezentarea, nu presupunerea că toate operațiile sunt la fel de sigure sau la fel de bine înțelese.

Obiectivul nu este să înlocuim orice componentă neuronală cu simboluri din principiu. Este să putem distinge ce a propus o componentă, ce a verificat alta, ce informație s-a păstrat și ce s-a aproximat. Dacă o reprezentare vectorială este utilă pentru selectarea unui circuit, poate fi folosită. Dacă răspunsul final trebuie să susțină o afirmație exactă, selectarea trebuie urmată de verificarea potrivită.

O reușită modestă, dar reală, ar fi un sistem care răspunde pe un domeniu restrâns, recunoaște ambiguitatea, produce justificări verificabile și se corectează local când apare o informație nouă. O reușită mai ambițioasă ar fi învățarea automată a reprezentărilor care permit această comportare pe domenii noi. Cartea nu confundă cele două trepte.

2. Ce numim înțelegere și ce nu pretindem despre existență

Patru angajamente care trebuie separate

Primul angajament este metodologic: căutăm modele explicite ale dependențelor dintre informații. Al doilea este inginerește verificabil: dorim comportamente robuste la reformulare, schimbarea premiselor și întrebări noi. Al treilea este filosofic: bănuim că înțelegerea presupune identificarea unor structuri stabile în variația experienței. Al patrulea ar fi ontologic: realitatea însăși este un circuit. Primele trei nu demonstrează al patrulea.

O hartă poate reprezenta relații spațiale fără ca orașul să fie făcut din hartă. Similar, o descriere computațională poate fi adecvată unui fenomen fără să epuizeze natura fenomenului. Pentru cercetarea propusă nu trebuie să stabilim dacă universul este discret, dacă mintea este integral computabilă sau dacă experiența subiectivă poate fi reprodusă. Trebuie să precizăm ce comportamente dorim să obținem și ce ar conta drept explicație a lor.

Această delimitare evită două extreme. Prima declară succesul înainte de a construi ceva, invocând universalitatea calculului. A doua respinge proiectul fiindcă nu rezolvă toate problemele filosofiei minții. Niciuna nu ajută experimentul. Putem cerceta competențe de înțelegere fără să pretindem că am rezolvat conștiința.

O definiție operațională, nu o esență finală

Vom spune că un sistem manifestă o competență de înțelegere într-un domeniu atunci când poate folosi o reprezentare a situației pentru a răspunde coerent unei familii de întrebări, a anticipa consecințe, a identifica informația lipsă și a-și revizui concluziile când se schimbă premisele. Definiția este deliberat relativă la domeniu și la probe. Nu este un verdict metafizic.

Un test nu trebuie să ceară doar reproducerea unui răspuns cunoscut. Putem schimba numele, ordinea propozițiilor, un interval temporal sau o condiție care pare minoră. Putem întreba de ce o concluzie nu rezultă. Putem introduce un fapt irelevant și observa dacă răspunsul se modifică nejustificat. Putem înlocui o premisă și verifica dacă sistemul retrage exact concluziile dependente de ea.

De exemplu, „Ana i-a împrumutat Mariei o carte” și „Ana i-a oferit definitiv Mariei o carte” pot avea aceeași consecință imediată asupra posesiei fizice, dar consecințe diferite asupra proprietății și obligației de returnare. O reprezentare bună pentru întrebarea „cine o are acum?” poate fi insuficientă pentru „cine o poate vinde legitim?”. A înțelege mai mult înseamnă uneori a păstra distincții pe care întrebarea inițială nu le cerea.

Semnificația are nevoie de un loc în lume

Un circuit poate fi corect față de o interpretare greșită. Dacă o propoziție ironică este tratată drept raport literal, inferențele ulterioare pot fi valide și totuși nepotrivite situației. Dacă senzorul este defect, calculul poate procesa impecabil date false. Trebuie deci distinse adevărul într-un model, fidelitatea interpretării textului și adecvarea modelului la lume.

Pentru SOP Lang, aceasta sugerează păstrarea separată a unui fragment sursă, a interpretării lui și a statutului epistemic al interpretării. „Documentul afirmă că ușa este închisă” nu este identic cu „ușa este închisă”. Prima afirmație poate fi verificată față de document; a doua cere încredere în document sau observație independentă. Proveniența permite auditarea acestei treceri, dar nu o face automat legitimă.

Nu există o singură scală a certitudinii care să substituie toate aceste distincții. Un scor de încredere în recunoașterea unei expresii nu este probabilitatea ca evenimentul descris să fi avut loc. O demonstrație formală nu este probabilitatea ca premisa ei să fie adevărată. O mașină de înțelegere trebuie să reprezinte ce fel de incertitudine are, nu doar câtă.

Ce ar fi un progres și ce ar fi doar o impresie

Un progres ar fi transferul unei reprezentări învățate către combinații neîntâlnite, cu justificări care rezistă verificării. Ar fi reducerea căutării fără eliminarea răspunsurilor valide. Ar fi o distincție nouă introdusă exact când un contraexemplu arată că vechea reprezentare era prea săracă.

O impresie de progres poate apărea când sistemul produce explicații convingătoare după ce a ghicit răspunsul, când testele seamănă prea mult cu exemplele de antrenare sau când costul construirii unei biblioteci este ascuns în munca umană. Aceste situații nu fac sistemul inutil. Dar trebuie descrise corect: asistență umană, memorare, verificare post-hoc sau generalizare reală sunt rezultate diferite.

Poziția filosofică de lucru a cărții este modest realistă: presupunem că există regularități care nu depind de dorința sistemului și că erorile pot fi descoperite prin confruntare cu date și argumente. Nu presupunem că orice regularitate are o descriere scurtă, că orice adevăr relevant este decidabil

sau că orice scop poate fi atins printr-un calcul fezabil. Programul caută insule de structură exploatabilă, apoi studiază cât de departe se pot extinde.

3. Ce este un circuit atunci când firele transportă lumi

Circuitul ca dependență, nu ca desen electronic

Un circuit de calcul este o organizare a operațiilor și a valorilor de care ele depind. O operație primește valori și produce valori. Un fir indică faptul că o valoare produsă într-un loc este folosită în altul. Nu este obligatoriu ca valoarea să fie un bit. Poate fi un număr, un vector, un tabel, un set de posibilități sau o descriere a altui circuit.

Dacă o funcție adună două prețuri, firele sale poartă numere. Dacă o operație leagă două tabele după identificatorul clientului, firele poartă relații. Dacă o operație interpretează o frază, ieșirea poate fi un set de structuri de evenimente. Același cuvânt, „fir”, descrie dependența logică dintre operații, nu materialul din care este realizată memoria.

Această generalitate trebuie controlată. Putem ascunde întreaga problemă într-un nod numit „înțelege totul”, dar atunci circuitul nu mai explică nimic. O descriere utilă precizează operațiile admise, contractele lor și costurile relevante. Nivelul de detaliu poate varia, însă o frontieră trebuie să separe mecanismele explicate de componentele încă tratate ca oracole.

SSA: nume stabile pentru rezultate

În forma SSA, fiecare nume de rezultat este atribuit o singură dată în reprezentarea statică. Când starea se schimbă, folosim o versiune nouă. În loc să spunem că același stoc este modificat de trei ori, distingem stocul inițial, stocul după re-

zervare și stocul după expediere. Astfel, o concluzie poate indica exact de ce versiune depinde. SSA este o tehnică consacrată a compilatoarelor; ea nu este, prin sine, o teorie a semnificației. [1, 2]

Buclele nu dispar prin această convenție. Un program poate repeta o operație de un număr necunoscut de ori, iar reprezentarea statică trebuie să descrie controlul repetiției și alegerea valorilor care vin pe căi diferite. Dacă desfășurăm o execuție finită, obținem o succesiune de versiuni concrete. Circuitul desfășurat și programul care poate genera execuții oricât de lungi sunt obiecte diferite.

Trebuie de asemenea să fim atenți la efecte. Două nume diferite care indică aceeași memorie modificabilă nu garantează independență. Pentru un model conceptual curat, efectele asupra mediului sunt reprezentate explicit: starea de dinainte, operația autorizată și starea rezultată. Imutabilitatea numelui nu face în mod magic imutabil tot ce poate fi accesat prin valoarea lui.

Trei obiecte care nu trebuie confundate

O sursă de circuit descrie cum se poate calcula. O instanță executată este aplicarea acelei descrieri la anumite intrări. Conținutul produs este valoarea obținută: de exemplu, un tabel de persoane și evenimente. Putem păstra toate trei, dar ele nu sunt interschimbabile.

O regulă generală despre transferul posesiei nu este același lucru cu episodul în care Ana îi dă cheia Mariei. Episodul nu este același lucru cu propoziția care îl descrie. Iar faptul materializat „Maria posedă cheia după transfer” nu este identic cu circuitul care l-a derivat. A păstra diferențele face posibilă reutilizarea regulii, corectarea episodului și reexecutarea numai a consecințelor afectate.

În SOP Lang, uniformitatea circuitelor ar trebui să însemne că aceste obiecte se pot conecta prin interfețe clare. Nu ar trebui să însemne că toate sunt amestecate într-un obiect opac numit cunoaștere. Un circuit poate produce alt circuit ca valoare, dar executarea circuitului produs rămâne o operație distinctă, cu propriile condiții.

Feedback, timp și procese deschise

Un sistem care conversează nu primește toate intrările de la început. Apar observații noi, obiectivele se schimbă, iar unele operații consultă mediul. Putem reprezenta fiecare etapă printr-o transformare a stării, dar nu trebuie să pretindem că știm anticipat întregul graf al viitorului.

O organizare pe epoci este o alegere posibilă: în interiorul unei epoci, anumite ipoteze și versiuni sunt fixate; o observație nouă deschide o epocă nouă, în care unele concluzii sunt păstrate, altele retrase. Aceasta este o propunere inginerească pentru audit, nu o condiție universală a inteligenței.

O circuitizare completă a unei execuții poate consuma multă memorie. Pentru cercetare, trebuie măsurat ce merită păstrat: fiecare operație elementară, numai inferențele relevante sau un certificat care permite verificarea rezultatului. Un jurnal enorm este transparent în principiu, dar poate rămâne inutil pentru un om. Auditabilitatea nu se obține doar prin refuzul de a șterge ceva.

4. Compoziționalitatea: de ce întregul nu trebuie să fie o surpriză totală

Ce știm când știm o componentă

Un programator folosește o bibliotecă fără să reanalizeze de fiecare dată toate instrucțiunile ei. Are nevoie de un contract: ce intrări sunt acceptate, ce rezultat se promite și ce efecte pot apărea. Compoziționalitatea este ideea că putem deduce proprietăți ale întregului din proprietățile componentelor și din felul conectării lor.

Să avem o operație care transformă grade Celsius în Fahrenheit și alta care compară temperatura cu un prag. Pentru compunere nu este suficient că ambele folosesc numere. Trebuie să știm unitatea și semnificația pragului. O interfață care spune doar „număr” este corectă ca tip de stocare și insuficientă ca tip semantic.

Același lucru apare în limbaj. Un tabel de evenimente poate conține momentul raportării sau momentul evenimentului. O operație temporală care le confundă poate fi perfect executabilă și conceptual greșită. Contractele trebuie să expună distincțiile care influențează continuarea.

Intuiția teoriei categoriilor

Teoria categoriilor oferă un vocabular abstract al compunerii. Putem privi tipurile de valori ca obiecte și transformările dintre ele ca săgeți. Dacă ieșirea unei transformări se potrivește cu intrarea alteia, le putem compune. Identitatea lasă valoarea neschimbată. Asociativitatea spune că gruparea formală a unei succesiuni compatibile nu îi schimbă transformarea totală. Teoriile monoidale adaugă un mod de a reprezenta calcule alăturate, nu numai în serie. [3]

Pentru cititorul inginer, câștigul nu este înlocuirea funcțiilor cu o terminologie solemnă. Este obligația de a spune ce structură se păstrează când trecem de la o reprezentare la alta. Dacă traducem un circuit de evenimente într-un circuit de constrângeri temporale, traducerea ar trebui să respecte compunerea relevantă. Altfel, fragmente corect traduse separat pot produce împreună alt sens.

Un exemplu elementar: o traducere care păstrează numai durata fiecărui segment de drum poate calcula durata totală a unui traseu dacă duratele se adună și nu există condiții de sincronizare. Dacă apar feriboturi cu orar, durata segmentelor nu mai este suficientă: trebuie păstrat și momentul sosirii. Eșecul nu este al compunerii ca idee, ci al interfeței alese pentru compunere.

Textul ca circuit are antecedente

Nu propunem că expresia „text ca circuit” începe aici. În DisCoCirc, Bob Coecke studiază o reprezentare compozițională în care propozițiile pot actualiza sisteme de semnificații, iar textul poate fi organizat circuital. Acest precedent este important tocmai fiindcă apropie structura textului de evoluția unor reprezentări, nu numai de clasificarea unor propoziții izolate. [4]

Pentru SOP Lang, întrebarea suplimentară este cum alegem și dezvoltăm automat circuitele, ce interfețe permit reunirea alternativelor și cum verificăm că o comprimare nu elimină o distincție necesară. Aceste obiective trebuie comparate cu lucrările existente, nu prezentate drept consecințe noi ale unei simple notații.

Ce nu rezolvă compoziționalitatea

Un joc de piese perfect compatibile poate avea un număr enorm de construcții. Faptul că știm ce se întâmplă după conectarea a două componente nu spune care două compo-

nente trebuie conectate. Teoria compunerii este o bază pentru verificare și organizare; algoritmul de căutare cere structură suplimentară.

Mai există o limită: semnificația contextuală poate depinde de lucruri aflate în afara componentelor selectate. Expresia „este cald” poate fi o descriere, o rugăminte de a deschide geamul sau o ironie. Un model poate trata acest context ca intrare suplimentară. Dar nu îl poate elimina doar fiindcă dorește o compunere elegantă.

Principiul de lucru devine astfel precis: când o compunere eșuează semantic, întrebăm dacă operația este greșită, dacă interfața este prea săracă sau dacă lipsește contextul. Cele trei explicații cer reparații diferite. Această diagnosticare este deja o contribuție practică a perspectivei circuitale.

PARTEA A II-A

Matematica informației parțiale

Un vocabular construit de la zero: posibilități, relații, rafinare și descrieri sigure ale stărilor.

5. Mulțimi, relații și întrebări despre lumi posibile

O valoare necunoscută nu este absența oricărei informații

Un program obișnuit primește adesea o valoare exactă: temperatura este 18. În raționament avem frecvent altceva: temperatura este între 15 și 20; persoana care a intrat este Ana sau Maria; evenimentul s-a petrecut după prânz, dar înainte de ședință. Putem calcula cu asemenea informații fără să inventăm prematur o valoare exactă.

O mulțime este aici doar o colecție de posibilități. Dacă un fir poate avea valorile 2, 3 sau 4, îl putem descrie prin mulțimea {2, 3, 4}. O informație nouă, „valoarea este pară”, lasă {2, 4}. Nu am ales încă valoarea, dar am progresat. Mulțimea vidă arată că informațiile simultan impuse nu pot fi satisfăcute.

Această interpretare este intuitivă și periculoasă dacă uităm ce conțin posibilitățile. Uneori ele sunt valori individuale. Alteori sunt stări complete ale lumii. Dacă spunem doar că Ana sau Maria a intrat și că Ana sau Maria a ieșit, nu știm dacă a fost aceeași persoană. Două mulțimi individuale pot ascunde o corelație decisivă.

Funcțiile transformă; relațiile permit

O funcție deterministă asociază fiecărei intrări admise o ieșire. O relație descrie combinații permise. Ecuația x + y = z poate fi folosită pentru a calcula z când știm x și y, dar poate fi folosită și pentru a restrânge x când știm ceva despre y și z. Relația nu privilegiază conceptual o singură direcție de utilizare.

Să știm că y este între 7 și 9, z este între 10 și 12, iar x între 0 și 100. Din relație rezultă că x trebuie să fie între 1 și 5. Nu am inversat o funcție pe o valoare exactă; am propagat o condiție necesară. Dacă mai aflăm că x este par, rămân 2 și 4. Dacă aflăm și z = 10, restricțiile pot fi propagate din nou.

O relație poate fi stocată explicit ca tabel de combinații, dar aceasta nu este singura variantă. Formula x + y = z este o descriere compactă a multor combinații. Un tabel de un milion de perechi egale poate fi înlocuit cu relația x = y, dacă domeniul și sensul egalității sunt cunoscute. Alegerea reprezentării influențează atât memoria, cât și operațiile ușor de efectuat.

„Există o posibilitate" nu înseamnă „rezultă concluzia"

Presupunem două interpretări posibile ale unui text. În prima, Ana este la gară. În a doua, Maria este la gară. Întrebarea „este posibil ca Ana să fie la gară?" are răspuns afirmativ. Întrebarea „știm că Ana este la gară?" nu are același răspuns.

O concluzie este susținută de toate posibilitățile rămase numai dacă este adevărată în fiecare dintre ele. O concluzie este compatibilă cu informația dacă este adevărată în cel puțin una. Aceste două criterii corespund intuițiilor „necesar în model" și „încă posibil în model". Un sistem care le confundă poate prezenta o ipoteză plauzibilă ca fapt derivat.

Există și cazul inconsecvenței: nu mai rămâne nicio lume posibilă. În logica clasică, anumite definiții ale implicației fac ca orice propoziție să fie consecință a unor premise incompatibile. Pentru o mașină de înțelegere, politica operațională trebuie să prevină prezentarea acestui fenomen ca succes. Rezultatul trebuie marcat „premise incompatibile", cu explicația conflictului, nu „orice răspuns este demonstrat".

Două feluri de combinare

Când două constrângeri sunt simultan valabile, intersectăm posibilitățile. „x este între 0 și 10” împreună cu „x este mai mare decât 6” restrânge domeniul. Când avem ipoteze alternative, reunim cazurile: „Ana a plecat” sau „Maria a plecat”. Reuniunea nu autorizează să luăm un efect din prima ipoteză și alt efect din a doua ca și cum ar fi coexistat.

Pentru limbaj, această regulă este crucială. Dacă într-o interpretare Ana primește cheia și în alta Maria o primește, nu putem concluziona că ambele persoane au primit-o. Trebuie păstrată identitatea cazului, explicit sau printr-o reprezentare comună care conservă dependențele. O reprezentare mai mică este utilă numai dacă nu creează lumi inexistente.

De aceea, o „stare a firului” poate fi un mic model relațional, nu doar o valoare cu un procent de încredere. Când ne întrebăm cum disciplinăm ieșirile SOP Lang, una dintre opțiuni este să cerem ca ele să precizeze dacă reprezintă fapte simultane, alternative, condiții sau rezultate exacte. Tipul epistemic al valorii contează la fel de mult ca tipul ei de date.

Exercițiu de intuiție

Trei cutii, A, B și C, conțin fiecare câte un obiect. Știm că exact una conține o cheie. Pentru fiecare cutie luată separat, valorile posibile sunt „cheie” și „nu cheie”. Dacă păstrăm numai aceste trei mulțimi individuale, permitem accidental și cazul cu trei chei, și cazul fără nicio cheie. Informația esențială este relația „exact una”.

Nu trebuie să enumerăm toate lumile pentru a păstra această relație. Putem reține o constrângere de cardinalitate. Acesta este primul exemplu al principiului general: reprezentarea bună nu este neapărat cea care conține mai puține simboluri, ci cea în care relația importantă rămâne ușor de folosit.

6. Ordini ale informației, rafinare și puncte fixe

Mai multă informație înseamnă mai puține posibilități

Dacă știm că un număr aparține intervalului [0, 100], iar apoi aflăm că aparține [20, 30], a doua descriere este mai informativă. Mulțimea posibilităților s-a micșorat. Vom numi acest proces rafinare. El nu trebuie confundat cu schimbarea lumii: poate că numărul a rămas același, iar noi am aflat mai mult despre el.

Există două convenții matematice frecvente. Putem ordona descrierile prin incluziunea mulțimilor de posibilități: mulțimea mică este sub mulțimea mare. Sau putem ordona prin cantitatea de informație: descrierea mai precisă este deasupra celei vagi. Sunt sensuri opuse ale aceleiași idei. Cartea va spune explicit ce se restrânge, pentru a nu ascunde convenția în simboluri.

În ordinea prin incluziune, universul tuturor posibilităților reprezintă necunoașterea, iar mulțimea vidă reprezintă incompatibilitatea. Nu sunt același lucru. „Nu știm care valoare este corectă” permite multe valori. „Nicio valoare nu satisface premisele” nu permite niciuna.

Ce este o structură de tip latice

O latice este, pentru scopul nostru, un spațiu de descrieri în care putem combina sistematic informații. Pentru mulțimi, intersecția găsește ce respectă simultan două condiții; reuniunea păstrează ambele familii de cazuri. Pentru intervale, intersecția rămâne interval sau devine vidă. Reuniunea a două intervale separate nu mai este întotdeauna un interval, așa că o reprezentare limitată la intervale poate folosi cel mai mic interval care le acoperă.

Dacă alternativele sunt x = 1 și x = 9, intervalul [1, 9] păstrează ambele valori, dar introduce și 2, 3 și altele. Aceasta este o aproximare. Nu este o eroare dacă știm ce garanție păstrează: nu pierde posibilități reale, însă poate include posibilități suplimentare. Mai târziu, un candidat găsit în aceste posibilități suplimentare trebuie verificat.

Avantajul unei structuri disciplinate este că știm cum se combină descrierile și ce fel de pierdere poate apărea. Nu mai lăsăm fiecare operație să inventeze arbitrar un format de ieșire. Interpretarea abstractă folosește asemenea structuri pentru a analiza programe prin descrieri aproximative ale stărilor lor. [5]

Propagarea informației

Imaginăm trei celule care conțin informație despre x, y și z, conectate de relația x + y = z. Când aflăm ceva nou despre oricare dintre ele, putem încerca rafinarea celorlalte. O rețea de propagatori generalizează această idee: componentele transmit consecințe, iar celulele combină informația primită. Lucrările lui Radul și Sussman oferă un model explicit al acestei organizări. [6]

Pentru un exemplu finit, fie domeniile inițiale x, y, z = {0, 1, 2}. Impunem x < y și y < z. Din prima relație eliminăm x = 2 și y = 0. Din a doua eliminăm y = 2 și z = 0. Revenind la prima, y = 1 obligă x = 0; revenind la a doua, obținem z = 2. O singură trecere nu a fost suficientă. Informația utilă a circulat prin rețea.

Punctul fix este starea în care o nouă aplicare a propagatorilor nu mai schimbă descrierile. Nu înseamnă automat că am găsit toate soluțiile sau că știm o valoare exactă. Înseamnă că regulile de propagare folosite nu mai pot adăuga informație în reprezentarea disponibilă.

Când se termină propagarea

Dacă domeniile sunt finite și fiecare schimbare elimină cel puțin o posibilitate, numărul schimbărilor stricte este finit. Dacă planificarea execuției nu ignoră la nesfârșit vreun propagator relevant, ajungem la stabilizare. Pentru operatori de rafinare monotoni, aplicați echitabil într-un cadru finit potrivit, putem obține închiderea comună determinată de acei operatori.

Această afirmație are premise importante. Dacă o componentă poate adăuga și elimina arbitrar aceeași informație, poate oscila. Dacă introduce permanent obiecte noi, spațiul nu mai este finit. Dacă unele operații se bazează pe absența provizorie a unui fapt, adăugarea ulterioară a acelui fapt poate cere retragerea unei concluzii. Atunci nu mai avem doar rafinare monotonă într-o stare fixată.

În domenii infinite, chiar o succesiune monotonă poate continua fără oprire. Analiza buclelor folosește uneori lărgirea controlată a descrierii, numită widening, pentru a forța stabilizarea. Prețul este precizia. Nu este o dovadă că procesul real se oprește, ci o tehnică prin care analiza produce o aproximare într-un timp controlabil. [5]

Legătura cu SSA și cu revizuirea cunoașterii

Rafinarea nu cere distrugerea vechii valori. Putem păstra descrierea inițială și crea o versiune nouă, mai precisă, legată de justificarea ei. Într-o urmă SSA, această organizare arată ce informație nouă a produs fiecare restrângere. Pentru execuție eficientă, un sistem poate folosi intern actualizări; interfața de audit poate expune versiuni. Alegerea concretă rămâne inginerească.

Revizuirea este altceva decât rafinarea. Dacă aflăm că o premisă a fost falsă, nu restrângem neapărat vechea mulțime de lumi; o înlocuim cu alta, care poate readmite posibilități

anterior eliminate. De aceea, originea fiecărei constrângeri trebuie păstrată. Fără proveniență, sistemul nu știe ce trebuie retras când o sursă este corectată.

Pentru SOP Lang, acest capitol sugerează un contract general: o operație trebuie să spună dacă produce o concluzie, rafinează o descriere, propune o alternativă sau revizuiește o ipoteză. Toate pot fi circuite, dar nu trebuie combinate ca și cum ar avea aceeași semnificație logică.

7. Interpretarea abstractă, explicată prin ceea ce alegem să uităm

De ce nu calculăm întotdeauna cu valorile exacte

Un analizor poate dori să știe dacă un program împarte vreodată la zero. Nu îl interesează fiecare rezultat numeric posibil, ci o proprietate relevantă. Dacă variabila este cu siguranță pozitivă, anumite verificări se simplifică. Dacă poate fi zero, analiza păstrează un avertisment. Ideea interpretării abstracte este să execute un calcul despre proprietăți ale valorilor, în loc să enumere toate valorile. [5]

Să alegem o abstracție extrem de simplă: paritatea. Numerele sunt rezumate prin „par” sau „impar”. Două numere impare adunate dau un număr par; un număr par și unul impar dau un număr impar. Pentru întrebări despre paritatea unei sume, rezumatul este suficient. Pentru întrebarea „suma este mai mare decât o sută?”, nu este.

Prin urmare, o abstracție nu este bună sau rea în mod absolut. Este potrivită sau nepotrivită unei familii de operații și întrebări. Această relativitate va deveni esențială pentru construirea circuitelor.

Două punți între concret și abstract

Notăm cu α operația care transformă o stare concretă în descrierea ei abstractă. De exemplu, α(7) = „impar”. Notăm cu γ interpretarea unei descrieri abstracte ca mulțime de posibilități concrete. Atunci γ(„impar”) este mulțimea numerelor impare. Literele nu ascund un mecanism misterios; denumesc cele două direcții ale legăturii.

O operație concretă F transformă valori. O operație abstractă F# transformă descrieri. Pentru siguranță, cerem ca toate rezultatele concrete posibile să fie incluse în descrierea abstractă produsă. Într-o notație compactă:

$F(\gamma(a)) \subseteq \gamma(F\#(a))$.

Formula spune: dacă pornesc de la orice valoare permisă de descrierea a și execut F, nu ajung în afara posibilităților anunțate de F#. O analiză poate anunța prea multe posibilități, dar nu are voie să le elimine pe cele reale atunci când promite această garanție.

Să folosim intervale. Pentru x în [2, 5], calculul 2x + 1 este cuprins în [5, 11]. Aici intervalul descrie exact rezultatele reale dacă x variază continuu. Pentru valori întregi, el admite și numere pare care nu apar; reprezentarea poate rămâne sigură, dar mai puțin precisă. Domeniul concret contează.

Siguranța nu este precizie

Dacă analiza spune „rezultatul poate fi între 0 și 100”, iar rezultatul real este 7, nu a greșit în sensul de mai sus. Poate fi totuși inutilă pentru obiectiv. Dacă întrebarea este „rezultatul este sub 10?”, descrierea nu decide. O analiză sigură poate răspunde prea des „nu știu”.

Cel mai sigur rezumat posibil, „orice se poate întâmpla”, este aproape inutil. De aceea cercetarea trebuie să măsoare simultan siguranța și puterea de discriminare. În cazuri con-

trolate, siguranța poate fi verificată exhaustiv. Puterea de discriminare se poate măsura prin numărul concluziilor decise, al candidaților eliminați corect și al alarmelor false rămase.

Un rezumat prea precis poate avea costul stării concrete. O reprezentare care păstrează toate lumile posibile nu pierde nimic, dar poate deveni exponențială. Întrebarea inginerească nu este „putem păstra totul?”, ci „ce structură ne permite să păstrăm exact relațiile importante la un cost acceptabil?”.

Exemplul în care intervalele uită prea mult

Fie x un număr între 0 și 100, iar y = x. Dorim să calculăm x − y. În realitate rezultatul este întotdeauna zero. Dacă păstrăm numai intervalele individuale x în [0, 100] și y în [0, 100], scăderea intervalelor produce [−100, 100]. Analiza este sigură, dar a uitat egalitatea dintre variabile.

Reparația nu este neapărat enumerarea valorilor lui x. Putem păstra relația x = y. Astfel, expresia se simplifică exact la zero. Acest exemplu arată de ce rafinarea tipului fiecărui fir nu este suficientă în general. Uneori trebuie îmbogățit limbajul relațiilor dintre fire.

O alegere posibilă este un domeniu de constrângeri liniare. Alta este păstrarea unor egalități simbolice. Fiecare permite anumite operații și impune anumite costuri. Nu există motiv să folosim aceeași abstracție pentru un calcul numeric, o relație de identitate și o ambiguitate de coreferință.

Exactitatea compozițională este o condiție mai puternică

Pentru anumite operații putem cere mai mult decât siguranța:

$\alpha(F(s)) = F^{\#}(\alpha(s))$.

În acest caz, rezumatul după operație poate fi calculat exact din rezumatul anterior. Pentru paritate și adunare, proprietatea funcționează. Pentru paritate și testul „mai mare decât o sută”, nu funcționează dacă vrem un răspuns exact, fiindcă numere cu aceeași paritate pot avea răspunsuri diferite.

Această condiție nu spune că α păstrează întreaga stare. Spune că informația eliminată nu mai este necesară operațiilor și observațiilor vizate. În aceste condiții putem folosi rezumatul ca stare a unei căutări comprimate. Vom dezvolta această idee în capitolul despre interfețele suficiente.

Interpretarea abstractă oferă deci două contribuții distincte proiectului: aproximări sigure pentru eliminarea unor ramuri imposibile și, în clase speciale, reprezentări care permit continuarea exactă fără starea completă. Prima contribuție este mai generală. A doua poate produce reduceri spectaculoase, dar cere demonstrații mai puternice.

Întrebarea de cercetare

SOP Lang ar putea asocia fiecărei familii de operații una sau mai multe interpretări abstracte. Aceeași operație de transfer poate actualiza simultan un tabel concret de evenimente, o descriere abstractă a posesiei și un rezumat al dependențelor de sursă. Expanderul ar folosi rezumatele ieftine pentru orientare și reprezentările mai bogate atunci când apare o distincție necesară.

Aceasta este o propunere, nu o proprietate obținută automat prin scrierea circuitelor. Trebuie verificat că interpretările sunt compatibile, că rezultatele nu sunt confundate și că trecerea la o reprezentare mai bogată nu pierde traseul de justificare. Un sistem cu multe abstracții necoordonate poate fi mai greu de înțeles decât unul cu o singură reprezentare simplă.

8. Corelațiile: informația care nu locuiește pe niciun fir singur

Aceleași valori locale, lumi diferite

Considerăm două situații cu două variabile binare. În prima, perechile posibile sunt (0, 0) și (1, 1). În a doua, perechile posibile sunt (0, 1) și (1, 0). Privită separat, fiecare variabilă poate fi 0 sau 1 în ambele situații. Privite împreună, situațiile sunt opuse: în prima variabilele sunt egale; în a doua sunt diferite.

Orice reprezentare care păstrează doar mulțimile individuale identifică aceste două situații. Dacă întrebarea finală privește egalitatea, identificarea este greșită. Acest mic exemplu conține esența problemei reprezentărilor distribuite: informația relevantă poate fi în relație, nu în componente luate separat.

Nu rezultă că avem nevoie obligatoriu de neuroni sau de vectori mari. Relația „egal” este o reprezentare simbolică foarte compactă a dependenței. Nu rezultă nici că simbolurile rezolvă orice dependență ieftin. Pentru anumite familii de relații, forma compactă poate fi greu de găsit sau greu de utilizat.

Un conflict invizibil descrierilor individuale

Fie x, y și z valori binare. Impunem că x și y sunt egale, că y și z sunt egale, dar că x și z sunt diferite. Fiecare variabilă, privită izolat, continuă să admită 0 și 1. Fiecare relație luată separat admite combinații. Împreună, relațiile sunt incompatibile.

Putem scrie egalitatea binară ca $x \oplus y = 0$, unde $\oplus$ înseamnă adunare modulo doi: $0 \oplus 0$ și $1 \oplus 1$ sunt zero, iar valorile diferite dau unu. Combinând cele trei ecuații, fiecare variabilă apare de două ori și se anulează. Rămâne $0 = 1$. Acesta este un certificat al incompatibilității.

Nu am explorat toate cele opt atribuiri pentru a obține explicația. Am folosit o algebră potrivită relației. În sisteme mai mari de asemenea ecuații, eliminarea liniară oferă un mecanism general de rezolvare. Cazul nu dovedește superioritatea oricărei abordări globale; arată insuficiența unui rezumat strict individual.

Frontiera unui circuit

Imaginăm că tăiem un circuit parțial în locul în care viitoarele operații se vor conecta. Valorile vizibile pe această tăietură formează frontiera. Pentru a decide ce continuări sunt posibile, putem avea nevoie de valorile lor, de relațiile dintre ele și de condițiile sub care au fost produse.

O frontieră nu este neapărat o listă mică. Dacă problema cere să comparăm toate persoanele între ele, relațiile pot crește repede. Totuși, anumite structuri permit rezumate compacte: o relație de ordine, o partiție în clase de egalitate, un sistem de ecuații sau un graf cu separatoare mici. Căutarea unei interfețe suficiente devine căutarea formei potrivite pentru această frontieră.

Un separator este un set de variabile prin care două regiuni ale problemei interacționează. Dacă știm suficient despre separator, putem trata regiunile mai independent. Intuitiv, nu trebuie să cunoaștem toate detaliile din două orașe pentru a calcula interacțiunea lor dacă toate schimburile relevante trec prin câteva porturi bine descrise. Condiția importantă este „toate schimburile relevante”. O legătură ascunsă invalidează separarea.

De ce „holografic” nu este încă un răspuns

O reprezentare distribuită poate codifica multe relații într-un singur vector. Aceasta schimbă modul accesului la informație, nu elimină nevoia de a o păstra. Dacă o operație de recuperare este aproximativă, trebuie măsurate confuziile. Dacă relațiile sunt suprapuse, trebuie înțeles ce interferențe apar.

La fel, un simbol compact poate reprezenta o relație foarte mare doar fiindcă interpretul său știe să o reconstruiască sau să o folosească. Complexitatea nu trebuie numărată numai în lungimea simbolului. Trebuie inclusă cunoașterea interpretului. Un identificator de un octet poate trimite la o bibliotecă uriașă; aceasta nu înseamnă că toată biblioteca a fost comprimată în opt biți.

Lecția pentru cercetare este clară: trebuie comparate informația disponibilă, costul accesării și tipurile de erori, nu doar dimensiunea aparentă a unei valori. Întrebarea „simbolic sau distribuit?” devine mai fertilă când o înlocuim cu „ce relații sunt reprezentate, ce operații le pot folosi și cu ce garanții?”.

PARTEA A III-A

Cum comprimăm construcția circuitului

De la arborele încercărilor la un graf partajat: condițiile în care două istorii pot avea același viitor relevant.

9. A executa, a verifica și a descoperi sunt probleme diferite

Circuitul cu fire perfecte poate rămâne greu de construit

Să disciplinăm radical ieșirile: fiecare fir transportă un singur bit, fiecare operație este o poartă logică bine definită, iar toate conexiunile sunt explicite. Putem evalua un asemenea circuit pentru o intrare dată urmărind operațiile. Dar întrebarea „există o intrare care produce unu?" poate necesita un efort foarte diferit.

Aceasta este o avertizare importantă pentru SOP Lang. Explozia combinatorială nu apare doar fiindcă valorile sunt texte vagi sau obiecte foarte bogate. Poate apărea și cu valori binare și reguli elementare. Un format disciplinat elimină anumite surse de confuzie, dar nu garantează o căutare eficientă în orice clasă de probleme.

Dacă un algoritm ar găsi în timp polinomial o intrare satisfăcătoare pentru orice circuit boolean, ar rezolva și formulările standard de satisfiabilitate codificate astfel. Cartea nu deduce de aici că proiectul trebuie abandonat. Deducem doar că o promisiune universală are consecințe teoretice mult mai puternice decât o optimizare inginerească obișnuită.

Lecția criptografiei

În criptografie, o transformare poate fi ușor de executat și dificil de inversat în condițiile presupuse de modelul de securitate. Cunoașterea algoritmului nu este, prin sine, o metodă eficientă de găsire a preimaginii. Funcțiile unidirecționale formalizează această asimetrie ca ipoteză de dificultate, nu ca adevăr demonstrat despre orice funcție aparent complicată. [7]

Pentru mașinile de înțelegere, analogia relevantă este limitată și precisă: transparența regulilor nu elimină automat dificultatea descoperirii intrării, a planului sau a circuitului potrivit. Nu afirmăm că interpretarea limbajului este o funcție criptografică. Folosim un exemplu consacrat în care direcția calculului contează fundamental.

Există și lecția pozitivă a certificatelor. Un rezultat poate fi însoțit de un obiect mai ușor de verificat decât de descoperit. Pentru un circuit SOP Lang, certificatul poate preciza premisele folosite, operațiile aplicate și condițiile îndeplinite. Dar certificatul validează o relație formală față de acele premise; nu dovedește că o sursă externă spune adevărul.

Patru locuri în care putem ascunde costul

Putem ascunde costul în bibliotecă: fiecare operație pare simplă fiindcă cineva a construit anterior un mecanism foarte puternic. Îl putem ascunde în reprezentare: o stare compactă trimite la un obiect enorm. Îl putem ascunde în preprocesare: interogarea este rapidă numai după o compilare costisitoare. Îl putem ascunde în evaluator: verificarea unui contract este ea însăși o problemă dificilă.

Un rezultat experimental trebuie să declare unde se află aceste costuri. Pentru un sistem folosit la un milion de întrebări, o compilare scumpă poate fi excelentă. Pentru o singură întrebare nouă, aceeași soluție poate fi neeconomică. Nu există contradicție; se schimbă regimul de utilizare.

De asemenea, un algoritm care folosește un tabel cu toate răspunsurile posibile poate avea interogări rapide. Acesta nu este, fără alte argumente, un algoritm eficient de învățare sau de construire a tabelului. Interfața suficientă ideală poate exista matematic și poate fi prea greu de obținut practic.

Ce fel de eficiență căutăm

Programul propus urmărește eficiență condiționată de structură. Pot exista domenii cu puține clase relevante de stări, relații cu algebră simplă, frontiere mici sau biblioteci reutilizabile. În asemenea domenii, comprimarea căutării poate fi decisivă. În alte domenii, sistemul trebuie să recunoască limitele bugetului și să ofere un rezultat incomplet explicit.

Un obiectiv rezonabil nu este „nu vom avea niciodată ramificare”, ci „nu vom ramifica separat istorii despre care putem demonstra că au același viitor relevant”. Această schimbare de formulare păstrează ambiția, dar elimină o pretenție nejustificată de a face ușor orice calcul.

Mai există un avantaj care nu se reduce la viteză: chiar atunci când căutarea rămâne scumpă, o reprezentare disciplinată poate face vizibil de ce este scumpă. Putem identifica relația care leagă multe variabile, ambiguitatea nerezolvată sau operația pentru care nu avem un rezumat suficient. Explicația costului este ea însăși un rezultat de cercetare.

10. Echivalența prin continuare: când două istorii sunt aceeași stare

Exemplul unui automat care citește cifre

Imaginăm un dispozitiv care citește un șir de cifre și trebuie să decidă la sfârșit dacă numărul reprezentat este divizibil cu trei. Nu trebuie să păstreze întregul șir. Poate păstra restul împărțirii la trei. Istorii foarte diferite ajung astfel în aceeași stare internă.

De ce este legitim? Dacă două prefixe dau același rest, orice sufix de cifre adăugat ambelor produce din nou același rest final. Niciun viitor permis nu mai poate distinge prefixele

pentru întrebarea vizată. Nu au același text, aceeași lungime sau aceeași valoare numerică. Au aceeași relevanță pentru continuare.

Această idee este centrală în teoria automatelor și în familia teoremelor Myhill–Nerode. Extensiile către automate cu registre și urme simbolice arată că echivalențele pot necesita păstrarea unor relații, nu doar a unei etichete finite simple. [8]

Echivalența contextuală

Pentru două circuite parțiale A și B, putem întreba dacă orice continuare admisă K produce aceeași observație relevantă atunci când este atașată lui A sau lui B. Scriem schematic:

$A \equiv_Q B$ atunci când, pentru orice K admis, $Obs(K[A]) = Obs(K[B])$.

Q indică familia întrebărilor sau observațiilor care contează. Dacă observăm numai rezultatul numeric, două circuite pot fi echivalente. Dacă observăm și consumul de energie, latența, efectele externe sau proveniența, aceeași echivalență poate dispărea. Nu există echivalență fără precizarea a ceea ce suntem dispuși să ignorăm.

Să luăm două trasee care ajung în aceeași gară la aceeași oră. Pentru continuarea unei călătorii, pot fi echivalente. Pentru decontarea cheltuielilor, pot fi diferite. Pentru o întrebare despre locurile vizitate, istoria este esențială. Același rezumat nu servește toate scopurile.

De ce aceasta este mai mult decât memorarea rezultatelor

Memorarea obișnuită reutilizează rezultatul unei funcții pentru aceeași intrare. Echivalența contextuală poate reuni intrări diferite și istorii diferite atunci când diferența lor nu influențează continuarea. Este o comprimare semantică a stării, nu doar o evitare a repetării identice.

Pentru suma unei secvențe, putem păstra suma parțială. Pentru media secvenței, suma singură nu ajunge: trebuie și numărul elementelor. Pentru mediană, suma și numărul nu sunt în general suficiente. Exemplele arată că dimensiunea stării necesare depinde de întrebarea viitoare și de operațiile permise.

Alegerea unui rezumat poate fi testată prin căutarea unei perechi de istorii pe care le reunește și a unei continuări care le desparte. O asemenea triplă — două istorii și o continuare — este un contraexemplu concret la suficiență. Nu trebuie să discutăm vag dacă rezumatul „pare să păstreze sensul”. Putem căuta situația exactă în care nu îl păstrează.

Similaritatea nu este echivalență

Două expresii pot avea rezultate apropiate pe exemplele văzute și rezultate diferite pe altele. Funcțiile $x + 1$ și $3x + 1$ coincid la $x = 0$, dar nu la $x = 1$. Un identificator semantic construit numai din primul exemplu le-ar reuni nejustificat.

Un vector de caracteristici sau o amprentă calculată pe un set de probe poate fi utilă pentru a găsi candidați la echivalență. Verificarea exactă poate veni după. Dacă domeniul este finit și testat integral, observațiile pot decide echivalența pe acel domeniu. Dacă domeniul este nelimitat, un număr finit de teste nu devine automat dovadă universală.

La fel, un hash al sursei detectează identitate sintactică cu riscurile tehnice cunoscute ale schemei folosite; nu demonstrează că surse diferite au aceeași semnificație. O mașină auditabilă trebuie să distingă egalitatea de reprezentare, egalitatea testată și egalitatea demonstrată.

Bisimularea și sistemele interactive

Pentru un proces care interacționează cu mediul, nu ne interesează doar un rezultat final. Două stări pot fi considerate echivalente dacă fiecare pas observabil dintr-una poate fi corespunzător unui pas din cealaltă, păstrând relația dintre stările rezultate. Aceasta este intuiția bisimulării.

Condițiile exacte depind de model: acțiuni alese de agent, acțiuni ale mediului, nedeterminism și probabilități nu sunt interschimbabile. Pentru SOP Lang, lecția practică este că o interfață potrivită unei funcții pure poate fi insuficientă unui agent care poate acționa. Trebuie păstrate și posibilitățile de interacțiune relevante.

Definiția prin toate continuările este un ideal semantic. Ea ne spune ce ar trebui să păstrăm. Nu ne dă încă o procedură eficientă pentru a decide echivalența. Următorul capitol caută condiții locale suficiente, verificabile pe operațiile bibliotecii.

11. Interfețele suficiente: o propoziție mică, cu consecințe mari

Contractul unui rezumat

Fie s starea completă a unei construcții parțiale și α(s) rezumatul ei. Pentru fiecare operație permisă F, presupunem că există o operație asupra rezumatelor, F#, astfel încât rezu-

marea după execuție coincide cu execuția asupra rezumatului. Presupunem și că răspunsul final poate fi calculat din rezumat, printr-o funcție h.

α(F(s)) = F#(α(s)); R(s) = h(α(s)).

Prima condiție spune că rezumatul poate fi continuat. A doua spune că el păstrează observația finală. Împreună, ele permit înlocuirea stării complete cu rezumatul pentru orice secvență de operații din biblioteca respectivă.

Argumentul este o inducție simplă. La început, rezumatul este corect prin definiție. Dacă înaintea unui pas rezumatul corespunde stării, prima condiție păstrează corespondența după pas. Repetând, corespondența rămâne valabilă până la final. Atunci a doua condiție produce răspunsul corect. Nu există aici un apel misterios la inteligență, ci o proprietate a reprezentării și a operațiilor.

Exemplul funcțiilor afine

O funcție de forma f(x) = ax + b poate fi reprezentată prin perechea (a, b). Dacă operațiile permise adună o constantă la rezultat sau înmulțesc rezultatul cu o constantă și apoi adună alta, funcția rămâne afină. Nu avem nevoie de lista tuturor operațiilor anterioare pentru a continua calculul funcțional.

De exemplu, dacă f(x) = 2x + 3 și adunăm 5, obținem 2x + 8. Dacă apoi înmulțim cu 3 și adunăm 1, obținem 6x + 25. Rezumatul se actualizează prin aritmetică asupra a două coeficiente. Istorii diferite care produc aceiași coeficienți au același comportament pentru continuările admise.

Dacă adăugăm însă operația „ridică rezultatul la pătrat”, familia afină nu mai este închisă. Perechea nu mai poate descrie exact orice rezultat. Putem îmbogăți reprezentarea că-

tre polinoame sau putem interzice operația în acest nivel al căutării. Aceasta este o decizie explicită, nu o excepție ascunsă.

Când arborele devine graf

Presupunem o căutare pe n etape. La fiecare etapă există cel mult M rezumate distincte și din fiecare se încearcă cel mult r extinderi. Dacă reunim toate construcțiile care au același rezumat la aceeași etapă, numărul evaluărilor de extinderi este cel mult de ordinul nMr.

Număr de extinderi evaluate $\leq n \cdot M \cdot r$.

Aceasta nu este o teoremă despre orice sinteză de programe. Presupune că starea rezumată conține tot ce influențează pașii viitori, inclusiv etapa sau bugetul atunci când contează. Presupune o bibliotecă precisă și o schemă de căutare stratificată. Costul evaluării, normalizării și comparării rezumatelor trebuie înmulțit separat. Pentru un graf arbitrar, nu putem înlocui fără justificare întreaga structură cu un număr de etapă.

Dacă M crește exponențial cu dimensiunea problemei, limita nu oferă eficiență polinomială. Dacă rezumatul trebuie descoperit printr-o căutare mai grea decât problema, avantajul online poate ascunde un cost anterior. Dacă numărăm toate căile, numerele rezultate pot avea tot mai mulți biți, iar operațiile asupra lor nu mai au cost constant.

De ce trebuie rezumată frontiera

O construcție poate produce două valori x și y. Viitoarea operație le poate compara. Atunci un rezumat separat al lui x și unul al lui y nu sunt suficiente dacă relația dintre ele s-a pierdut. Starea α(s) trebuie să reprezinte frontiera comună, în contextul continuărilor permise.

Mai trebuie inclus ceea ce influențează legalitatea continuării. Dacă o operație consumă o resursă, două circuite cu aceeași ieșire, dar cu resurse rămase diferite, nu sunt echivalente pentru toate planurile viitoare. Dacă o concluzie trebuie susținută de o anumită clasă de surse, proveniența relevantă nu poate fi eliminată doar pentru că valoarea calculată coincide.

Putem totuși separa nivelurile. Pentru orientarea unei căutări, poate ajunge echivalența valorilor. Pentru emiterea rezultatului, păstrăm un martor și verificăm proveniența. Dar dacă alegerea viitoare depinde chiar de proveniență, aceasta trebuie inclusă în starea folosită de căutare. Altfel comprimarea poate elimina singura variantă acceptabilă.

Programul de cercetare ascuns în propoziție

Matematica de bază nu este nouă: se înrudește cu programarea dinamică, omomorfismele și congruențele semantice. Posibila contribuție a SOP Lang ar fi identificarea unor asemenea interfețe pentru familii utile de raționamente și învățarea lor din texte, exemple și contraexemple.

Ar trebui cercetat dacă un sistem poate începe cu o reprezentare bogată, poate observa regularități între continuări și poate propune un rezumat mai mic. Apoi un verificator ar încerca să găsească un contraexemplu. Un rezumat acceptat ar putea deveni interfața unui circuit reutilizabil. Repetând, sistemul ar învăța nu doar programe, ci forme de stare în care programele devin mai ușor de construit.

Aceasta este ipoteza cea mai ambițioasă a cărții. Nu este confirmată de demonstrarea unei proprietăți pentru o reprezentare furnizată de om. Tocmai distanța dintre „există un rezumat suficient” și „sistemul îl descoperă la cost util” trebuie să organizeze experimentele.

12. Programarea dinamică, compilarea cunoașterii și grafurile de alternative

Aceeași subproblemă, mai multe drumuri către ea

Programarea dinamică evită rezolvarea repetată a unor subprobleme echivalente. Pentru a găsi un drum ieftin, de exemplu, nu păstrăm neapărat toate traseele care ajung într-un nod. În condiții potrivite, păstrăm costul minim relevant și un predecesor. Dar aceste condiții contează: dacă obiectivul cere să nu repetăm niciun oraș, istoricul vizitelor poate deveni parte a stării.

Aceeași lecție se aplică circuitelor. Nu putem reuni construcții doar pentru că produc aceeași valoare acum. Trebuie să știm că trecutul eliminat nu mai influențează legalitatea sau valoarea viitorului. Programarea dinamică reușește când o stare bine aleasă face subproblemele independente de detalii inutile ale istoriei.

Compilarea cunoașterii

Putem transforma o bază de cunoștințe într-o reprezentare în care anumite întrebări se răspund mai ușor. Compilarea poate fi scumpă, dar repetarea interogărilor poate justifica investiția. Harta lui Darwiche și Marquis compară tocmai compromisurile dintre compactitatea reprezentărilor, interogările eficiente și transformările disponibile. [10]

Nu există un format universal câștigător. O structură poate permite verificarea rapidă a compatibilității și poate face dificilă actualizarea. Alta poate fi compactă pentru o familie de formule și foarte mare pentru alta. Alegerea trebuie făcută pornind de la întrebările și modificările anticipate.

Pentru SOP Lang, o carte sau un dosar poate fi compilat într-un graf de entități, evenimente și justificări. Dar costul ingestiei, corectarea interpretărilor și adaptarea la întrebări

neprevăzute trebuie incluse în evaluare. Un rezumat perfect pentru întrebările de astăzi poate fi o arhivă proastă pentru cercetarea de mâine.

Lățimea relațiilor

În multe probleme, dificultatea depinde de câte variabile trebuie tratate împreună într-un pas de eliminare sau de separare. Noțiunea de treewidth formalizează un aspect al acestei lățimi. Descompunerile AND/OR și AOMDD-urile exploatează independențe condiționate pentru reprezentări și calcule mai compacte. [11]

Intuitiv, dacă două regiuni mari interacționează numai prin două variabile, putem rezuma fiecare regiune față de acele două variabile. Dacă interacțiunea trece prin aproape toate variabilele, rezumatul poate deveni enorm. Numărul variabilelor totale nu este singurul indicator al dificultății; forma conectării lor contează.

Totuși, lățimea grafului nu epuizează structura exploatabilă. Relațiile pot avea proprietăți algebrice sau dependențe funcționale care reduc calculul chiar când graful pare dens. Lucrările asupra eliminării variabilelor în prezența dependențelor funcționale ilustrează această posibilitate. [12] Sistemul trebuie deci să poată recunoaște atât separarea structurală, cât și algebra potrivită.

E-graphs: multe expresii egale, un singur spațiu partajat

Un e-graph păstrează familii de expresii despre care regulile acceptate spun că sunt echivalente. În loc să alegem imediat între a + 0 și a, le putem reprezenta în aceeași clasă și continua explorarea. Egalitatea este propagată în contexte compatibile. Ulterior extragem o expresie potrivită unui criteriu de cost. Sistemul egg este un reper practic pentru această organizare. [17]

Aceasta diferă de păstrarea unor interpretări alternative. Expresiile din aceeași clasă de egalitate trebuie să fie egale în semantica și sub ipotezele folosite. Două interpretări ale unui pronume nu sunt automat egale; sunt alternative care pot produce răspunsuri diferite. Ambele pot fi partajate într-un graf, dar partajarea nu autorizează confundarea sensului lor.

Regulile de rescriere trebuie și ele verificate. Transformarea unei împărțiri x/x în 1 cere o condiție precum x diferit de zero într-o semantică numerică potrivită. Cu efecte sau valori speciale, chiar rescrieri familiare pot cere precauții. O bibliotecă de egalități neverificate transformă comprimarea într-o sursă de erori sistematice.

Comprimarea nu elimină orice explozie

Un graf partajat poate deveni foarte mare. Equality saturation are nevoie uneori de ghidare pentru a ajunge la optimizări complexe în bugete practice; cercetările asupra ghidării prin schițe studiază explicit acest aspect. [30] A spune „folosim un e-graph” nu este un răspuns complet la problema controlului căutării.

SOP Lang ar putea combina reprezentări: clase de egalitate pentru expresii demonstrabil identice, grafuri împachetate pentru interpretări alternative și abstracții pentru filtrare. Important este ca fiecare legătură să aibă un sens declarat. Egalitate, posibilitate, implicație și similaritate nu trebuie să fie patru etichete pentru aceeași muchie ambiguă.

13. Tipuri, cerințe inverse și rafinare prin contraexemple

Tipurile ca prime filtre

Un tip poate spune că o operație primește o persoană și un eveniment, nu două numere. Un tip mai bogat poate spune că evenimentul este un transfer și că persoana este destinatarul lui. Tipurile rafinate adaugă condiții logice proprietăților de bază. În sinteză, ele pot elimina compuneri incompatibile înainte de execuția concretă. Cercetările asupra sintezei din tipuri rafinate explorează această direcție. [13]

Totuși, compatibilitatea tipurilor nu înseamnă că operația este utilă scopului. Mii de operații pot primi și produce texte. O disciplină care vede doar tipul „text” nu orientează suficient căutarea. Avem nevoie de contracte care spun ce relație există între intrare și ieșire.

Propagarea cerințelor înapoi

Dacă obiectivul cere un rezultat pozitiv și operația propusă calculează $x - 5$, o cerință necesară este $x > 5$. Dacă obiectivul cere identificarea posesorului unui obiect după un transfer, operația are nevoie de identitatea obiectului, a destinatarului și de condițiile temporale relevante. Cerințele rezultate pot deveni subprobleme.

Pentru o operație deterministă, precondiția cea mai slabă a unui obiectiv este mulțimea intrărilor care garantează obiectivul după execuție. Nu trebuie să o putem calcula exact în toate cazurile. O condiție necesară poate fi suficientă pentru a respinge anumite ramuri. O condiție suficientă poate permite acceptarea sigură a altora. Cele două direcții de aproximare trebuie marcate distinct.

Un expander poate întâlni înainte ce este posibil și înapoi ce este necesar. Dacă cele două nu au intersecție, ramura este imposibilă în model. Dacă au intersecție, ramura rămâne candidată. Intersecția nevidă nu este dovada existenței unei execuții concrete care satisface toate relațiile pierdute prin aproximare.

CEGAR: când o alarmă arată limita modelului

Să presupunem că analiza intervalelor vede x și y în [0, 100] și semnalează că x − y ar putea fi negativ. Modelul concret conține însă y = x. Contraexemplul abstract, de exemplu x = 0 și y = 100, nu este realizabil. În loc să declarăm programul greșit, îmbogățim abstracția cu relația lipsă.

Aceasta este intuiția CEGAR: începem cu un model abstract, verificăm un candidat la contraexemplu și rafinăm modelul când contraexemplul este fals. Lucrarea fondatoare formulează acest ciclu pentru verificarea sistemelor. [16] În SOP Lang, un mecanism înrudit ar putea introduce o distincție semantică exact atunci când absența ei produce o derivare suspectă.

Nu orice rafinare trebuie acceptată. Una care adaugă aproape întreaga stare concretă poate rezolva cazul curent și distruge avantajul. Trebuie măsurat câte erori elimină și cât crește reprezentarea. Rafinarea este o investiție, nu un bine fără cost.

CEGIS: când un candidat la program este greșit

În sinteza ghidată de contraexemple, construim un candidat care satisface exemplele curente. Un verificator caută o intrare pe care candidatul încalcă specificația. Dacă o găsește, adăugăm cazul și construim alt candidat. Aici reparăm programul candidat, nu neapărat abstracția analizei. Sinteza ghidată de sintaxă separă explicit gramatica programelor admise de specificația comportamentului. [15]

Distincția dintre CEGAR și CEGIS este utilă practic. Dacă o interpretare falsă este admisă fiindcă rezumatul a pierdut o relație, problema este în abstracție. Dacă rezumatul este adecvat, dar circuitul ales calculează altceva, problema este în candidat. Dacă textul a fost interpretat greșit de la început, ambele mecanisme pot verifica impecabil problema greșită.

Un ciclu pentru învățarea interfețelor

Putem propune un al treilea obiect al rafinării: chiar relația prin care reunim stările. Sistemul propune că două frontiere sunt echivalente. Un verificator caută o continuare care le distinge. Când o găsește, adăugăm caracteristica lipsă în interfață. Când nu o găsește într-un buget limitat, rezultatul trebuie etichetat „nu s-a găsit contraexemplu”, nu automat „echivalență demonstrată”.

TYGAR este un precedent pentru folosirea rafinării abstracțiilor ghidate de tipuri în controlul sintezei. [14] Cercetarea propusă extinde întrebarea către reprezentări semantice ale situațiilor și ale frontierelor circuitelor, fără a presupune că transferul este automat.

Un succes ar fi descoperirea repetată a unor interfețe mici care generalizează la probleme noi. Un eșec informativ ar fi situația în care fiecare contraexemplu cere o excepție nouă, iar reprezentarea crește aproape proporțional cu toate cazurile memorate. Acesta ar arăta că nu am găsit încă structura potrivită sau că familia de probleme nu o oferă în forma căutată.

PARTEA A IV-A

Limbaj, memorie și reprezentări distribuite

Ce trebuie păstrat din text, din incertitudine și din memorie pentru ca întrebările noi să rămână posibile.

14. De la propoziții la circuite de situație

Textul nu este încă baza de cunoștințe

O propoziție este o sursă de informație, nu un tabel deja validat de fapte. „Maria crede că Ana a plecat” conține un raport despre o credință. Nu autorizează automat introducerea faptului „Ana a plecat” în descrierea lumii acceptate. „Ana ar fi plecat dacă venea trenul” cere altă structură. Un sistem care extrage numai nume și verbe poate pierde tocmai diferența esențială.

În SOP Lang, ingestia ar trebui să construiască circuite care propun și verifică interpretări, iar execuția lor să producă reprezentări materializate. Putem avea tabele pentru entități, evenimente, roluri, relații temporale și surse. Aceste tabele sunt valori ale circuitelor. Nu trebuie confundate cu sursa circuitului care le produce.

Un episod de transfer poate avea un identificator de eveniment, un agent, un destinatar, un obiect, un interval temporal și o sursă textuală. Referințele către persoane și obiecte funcționează conceptual ca legăturile dintre tabelele unei baze de date. Aceasta permite ca două propoziții să se refere la același obiect fără să repetăm toată descrierea lui.

Un exemplu complet, dar mic

Considerăm textul: „Ana avea cheia luni dimineață. La prânz i-a dat-o Mariei. Seara, Maria a intrat în bibliotecă.” O reprezentare inițială ar identifica două persoane, o cheie, o bibliotecă, o stare de posesie și două evenimente. Pronumele din a doua propoziție trebuie legate de cheia și de persoanele relevante.

Pentru întrebarea „cine avea cheia după prânz?”, circuitul ar folosi regula de transfer și ordinea temporală. Pentru „cine deținea proprietatea asupra cheii?”, textul poate fi insuficient: posesia fizică nu implică neapărat proprietatea. Pentru „a folosit Maria cheia ca să intre?”, intrarea în bibliotecă nu dovedește singură mecanismul deschiderii ușii.

Aceste diferențe ar trebui să apară în structura reprezentării, nu doar într-o avertizare verbală adăugată la final. Un fapt despre posesie, o presupunere despre instrument și o întrebare fără răspuns au statute distincte. O redare fluentă nu trebuie să le niveleze.

Regulile ca circuite reutilizabile

O regulă de transfer poate primi un eveniment validat și o stare anterioară, apoi poate produce o stare ulterioară a posesiei. Regula poate cere condiții: obiectul există, participanții sunt identificați, evenimentul este actual și nu doar ipotetic, iar momentul relevant este stabilit suficient. Aceste cerințe fac regula selectabilă și verificabilă.

O asemenea regulă nu trebuie tratată drept lege universală a limbajului sau a lumii. „Ana i-a dat o idee Mariei” nu transferă posesia în același sens ca o cheie. „I-a dat cheia, dar a retras-o imediat” cere un eveniment ulterior. Un contract adecvat definește domeniul în care regula este folosită și permite abținerea în afara lui.

Într-un sistem care învață, unele circuite pot fi generate inițial de un model lingvistic sau de un cercetător. Acest fapt trebuie declarat. Faptul că rezultatul este un circuit auditabil nu înseamnă că sistemul a învățat autonom regula. Cercetarea trebuie să distingă generarea, validarea și reutilizarea regulilor.

Datalog ca intuiție a cunoașterii derivabile

Datalog oferă un model simplu: pornim de la relații de bază și de la reguli care produc relații noi. Dacă A este părinte al lui B și B este părinte al lui C, putem deriva că A este bunic al lui C. Repetăm aplicarea regulilor până când nu mai apar fapte noi. Tutorialul oficial Soufflé arată această organizare relațională. [20]

Pentru reguli pozitive asupra unui univers finit, calculul poate fi înțeles ca atingerea unui punct fix. Nu trebuie să alegem o singură cale de demonstrație ca să știm că faptul este derivabil. Putem păstra mai multe justificări și reutiliza subderivări comune.

Totuși, aritmetica nelimitată, generarea de termeni noi, negarea și efectele externe schimbă condițiile de terminare și semantica. Datalog nu este sinonim cu întregul limbaj natural. Pentru SOP Lang, poate inspira sau realiza o clasă de operații de derivare, fără să devină o explicație universală a interpretării.

Uniformitate fără un al doilea limbaj ascuns

Ambiția SOP Lang este ca interpretările, regulile și strategiile să poată fi exprimate prin circuite compozabile. Aceasta nu cere dispariția oricărei primitive. Un sistem are nevoie de operații de bază a căror semnificație este stabilită în afara propriei desfășurări. Important este ca această bază să fie explicită și suficient de mică pentru a putea fi analizată.

Dacă majoritatea raționamentului real se află într-un interpret opac, iar circuitul doar îi transmite text, uniformitatea devine cosmetică. Dacă, dimpotrivă, circuitele expun entități, condiții, alternative și justificări, ele devin obiecte ale cercetării. O probă utilă este să întrebăm ce poate verifica un cercetător despre un rezultat fără să creadă pe cuvânt componenta care l-a propus.

15. Ambiguitatea fără multiplicarea tuturor lumilor

De ce alegerea imediată poate fi o greșeală

Textul „Ana îi dă cheia Mariei. Ea pleacă la gară” admite, în lipsa unui context suplimentar, cel puțin două interpretări ale pronumelui. Pentru întrebarea „cine are cheia după transfer?”, ambele pot produce același răspuns, sub modelul explicit în care transferul schimbă posesia, iar plecarea nu o schimbă. Pentru „cine pleacă?”, diferența este esențială.

Un sistem care decide imediat pronumele poate introduce o eroare inutilă. Un sistem care dezvoltă integral două lumi separate poate dubla calcule care nu depind de alegere. Soluția conceptuală este păstrarea unei alternative locale și partajarea structurii comune. Rezolvăm ambiguitatea când obiectivul o cere sau când apar informații relevante.

Aceasta nu înseamnă că orice ambiguitate poate fi ignorată până la final. Unele decizii influențează tipul evenimentului, identitatea obiectelor și aplicabilitatea regulilor. Atunci trebuie menținute condițiile care leagă consecințele de fiecare alternativă. Partajarea este corectă numai dacă aceste dependențe rămân recuperabile.

Intuiția analizării prin chart

În analiza sintactică, aceeași porțiune de text poate participa la mai multe interpretări. Un chart păstrează rezultate parțiale pentru intervale ale textului, astfel încât substructurile comune să nu fie reconstruite la fiecare utilizare. Un graf împachetat poate descrie mulți arbori de analiză fără să îi enumere separat.

Pentru gramatici independente de context fixate, algoritmii clasici pot avea un număr pătratic de celule și un număr cubic de combinații în lungimea textului, cu factori dependenți de gramatică. Această proprietate nu se transferă automat la semantica nelimitată, coreferință sau cunoaștere despre lume. Totuși, mecanismul partajării rămâne instructiv. Tratarea prin semiringuri a lui Goodman arată cum aceeași structură de analiză poate susține calcule diferite. [18]

Pentru SOP Lang, intervalele pot produce circuite candidate, nu doar etichete gramaticale. Două expresii care descriu același eveniment pot reutiliza fragmente. Dar dacă echivalența lor este doar presupusă de un model, ea nu trebuie confundată cu o egalitate verificată.

Ce este un semiring și de ce contează

Un semiring oferă două operații: una pentru alternative și una pentru combinarea componentelor unei derivări. În cazul existenței unei analize, alternativele se combină prin „sau”, iar componentele necesare prin „și”. Pentru numărarea derivărilor, adunăm alternativele și înmulțim numărul combinațiilor independente. Pentru cost minim, alegem minimul între alternative și adunăm costurile componentelor.

Același graf poate astfel răspunde la întrebări diferite: există o derivare, câte derivări sunt, care este cea mai ieftină? Dar fiecare calcul are o semantică precisă. Două derivări ale aceleiași concluzii nu sunt neapărat două lumi distincte. Numărul derivărilor nu este automat o probabilitate.

Dacă o componentă comună apare în două ramuri, nu putem trata fără analiză cele două apariții ca dovezi independente. Pentru probabilități, dependențele și suprapunerea evenimentelor trebuie modelate. O sumă a unor scoruri poate fi o euristică utilă, dar nu trebuie etichetată probabilitate calibrată fără un model adecvat.

Proveniența ca expresie a dependențelor

Putem nota sursele prin simboluri p, q și r. Dacă o concluzie necesită p și q, proveniența poate conține produsul pq. Dacă rezultă fie din p și q, fie din r, putem păstra o expresie pq + r. Semnele descriu aici două moduri de construire a justificării, nu aritmetica obișnuită a adevărului.

Semantica provenienței prin semiringuri dezvoltă această idee pentru logici și baze de date. Extensiile către negație și puncte fixe arată că proprietățile algebraice trebuie alese cu grijă, nu extrapolate mecanic din cazul pozitiv simplu. [19, 32]

Pentru un cercetător, câștigul este concret: când sursa p este retrasă, vedem dacă r susține încă rezultatul. Când două surse par diferite, dar provin din același document, putem evita interpretarea lor ca sprijin independent. Când un răspuns are o justificare circulară, structura derivării poate face cercul vizibil.

Ce comprimăm și ce păstrăm

Un graf împachetat nu trebuie să păstreze toate detaliile fiecărei interpretări în fiecare nod. Poate folosi referințe comune, condiții de ramură și operații de reconstrucție. Însă trebuie să existe o cale de a verifica faptul că un răspuns provine dintr-o interpretare coerentă, nu din combinarea arbitrară a fragmentelor mai multor interpretări incompatibile.

Ipoteza de cercetare este că multe ambiguități naturale sunt irelevante pentru multe întrebări și că sistemul poate exploata această structură. Nu presupunem că toate sunt locale sau ieftine. Un experiment bun va crește separat numărul ambiguităților și numărul relațiilor care le leagă. Zece ambiguități independente și zece ambiguități global corelate sunt probleme diferite.

16. Timp, negație, cuantificatori și cauzalitate

Timpul nu este un atribut decorativ

„Maria are cheia” și „Maria avea cheia ieri” nu pot fi introduse ca același fapt. O stare este valabilă într-un interval sau într-un context temporal. Un eveniment poate schimba starea. Un raport poate fi făcut mult după eveniment. Aceste trei momente — valabilitate, producere și raportare — trebuie distinse când influențează răspunsul.

Un model simplu poate folosi intervale și relații precum înainte, după, simultan sau inclus în. Nu trebuie să cunoască întotdeauna o dată exactă. Din „transferul a avut loc înaintea plecării” și „plecarea înaintea închiderii” poate deriva ordinea fără să inventeze ore.

Persistența este o regulă de model: o stare continuă până când un eveniment relevant o schimbă. Nu este o consecință automată a oricărui text. Dacă nu știm ce s-a întâmplat între două momente, o presupunere de persistență trebuie marcată și eventual revizuită. În domenii diferite, regulile de persistență diferă.

Negația nu este lipsa unei înregistrări

Dacă baza nu conține faptul că Ana este în bibliotecă, nu rezultă în general că Ana nu este acolo. Absența unei dovezi și dovada absenței sunt lucruri diferite. Într-o bază adminis-

trativă completă pentru un scop precis, putem adopta o convenție de lume închisă. În lectura unor texte fragmentare, această convenție este adesea nepotrivită.

Sistemul ar trebui să distingă „susținut”, „infirmat”, „nedecis” și „incompatibilitate între surse”, dacă acestea sunt stările epistemice ale aplicației. Nu este obligatoriu ca toate să fie reprezentate printr-o anumită logică cu patru valori. Este obligatoriu să nu fie confundate în răspuns.

Negația schimbă și monotonia. Dacă o regulă spune „presupune disponibil dacă nu apare rezervat”, adăugarea ulterioară a unei rezervări retrage concluzia. Circuitul trebuie să păstreze dependența de această absență controlată. Nu poate trata rezultatul ca pe un fapt pozitiv derivat definitiv.

Cuantificatorii schimbă ce trebuie păstrat

„Fiecare cercetător a citit o carte” poate însemna că fiecare a citit câte o carte, eventual diferită. Nu implică faptul că există aceeași carte citită de toți. Ordinea lui „pentru fiecare” și „există” schimbă sensul.

O reprezentare care extrage doar relația citește(cercetător, carte) poate pierde domeniul și dependența alegerii. Pentru unele întrebări, informația este suficientă: fiecare cercetător a citit ceva. Pentru altele, precum organizarea unui seminar despre o carte comună, nu este. Acesta este încă un caz de interfață suficientă pentru o întrebare și insuficientă pentru alta.

Comparativele și condiționalele adaugă dificultăți asemănătoare. „Mai rapid decât” cere un termen de comparație și un criteriu. „Dacă plouă, ședința se mută” nu afirmă că plouă și nici că ședința s-a mutat. Circuitele trebuie să reprezinte relațiile condiționale fără a materializa prematur consecințele.

Raportarea și punctele de vedere

„Ana spune că Maria a plecat” produce cel puțin un fapt despre actul de raportare. Poate produce și o ipoteză despre plecare, cu sursa și statutul potrivit. Dacă Maria spune contrariul, conflictul poate exista între rapoarte despre lume, fără ca vreunul dintre actele de raportare să fie fals descris.

O reprezentare cu contexte sau lumi indexate permite păstrarea acestei structuri. Nu trebuie ca orice conversație să genereze un univers filosofic complet. Este suficient ca afirmațiile să poarte contextul care le schimbă folosirea: acceptat de sistem, raportat de o sursă, presupus într-un scenariu sau negat explicit.

Dependența de calcul nu este automat cauzalitate

Într-un circuit, o ieșire depinde de intrări prin construcție. Într-un model al lumii, o corelație între două variabile nu spune singură ce se întâmplă dacă intervenim asupra uneia. Un sistem poate prezice umbrela din ploaie și ploaia din umbrelă, dar deschiderea umbrelei nu produce în același sens ploaia.

Modelele cauzale structurale studiază mecanisme și intervenții, nu doar condiționări observaționale. Pearl explică distincția dintre a observa o valoare și a modifica mecanismul care o produce. [27] Pentru SOP Lang, un arc cauzal trebuie să provină dintr-un model sau dintr-o ipoteză cauzală declarată, nu doar din ordinea operațiilor de procesare a textului.

Un experiment conceptual util ar construi două lumi cu aceleași regularități observaționale în datele disponibile, dar răspunsuri diferite la intervenție. Dacă sistemul le confundă, reprezentarea predictivă nu este suficientă pentru întrebări cauzale. Aceasta nu o face inutilă; îi delimitează domeniul.

17. Simbolic, neuronal, distribuit: trei axe, nu două tabere

Un vector poate fi valoarea unui fir

O rețea neuronală poate fi descrisă ca un graf de operații asupra unor tensori. Un tensor este o colecție organizată de numere. Un fir conceptual poate transporta întregul tensor sau putem descrie separat coordonatele lui. Schimbarea granularității nu elimină caracterul distribuit al reprezentării. Operațiile unui Transformer sunt un exemplu consacrat de calcul exprimabil prin asemenea dependențe. [21]

Prin urmare, faptul că o valoare are un nume SSA nu înseamnă că semnificația este localizată într-un singur simbol. Un identificator poate denumi un vector în care informația este distribuită. Invers, o relație simbolică poate depinde de mai multe obiecte și de context. Localizarea numelui nu este localizarea sensului.

Trebuie separate cel puțin trei axe: reprezentări discrete sau continue; semnificații explicite sau învățate implicit; operații exacte sau aproximative. Un sistem poate fi simbolic și probabilistic, vectorial și determinist sau hibrid. O clasificare binară pierde aceste combinații.

De ce reprezentările distribuite pot fi utile

O reprezentare distribuită poate face ca obiecte asemănătoare să aibă descrieri apropiate și poate permite recuperarea după indicii incomplete. Într-un sistem de selecție a circuitelor, acest lucru poate fi foarte util: problema nu trebuie să coincidă textual cu descrierea operației ca să fie găsit un candidat relevant.

VSA/HDC explorează operații asupra reprezentărilor de dimensiune mare. O operație de legare poate asocia un rol cu un conținut, iar una de agregare poate reuni mai multe ele-

mente într-o reprezentare. Cercetarea asupra reprezentărilor holografice reduse generalizate investighează inclusiv proprietăți ale legării și compoziționalității. [22]

Pentru intuiție, putem imagina o reprezentare care combină „agent: Ana”, „acțiune: oferă” și „obiect: cheie”. Dorim să putem întreba cine este agentul sau ce obiect apare. Dacă recuperarea este aproximativă, trebuie măsurat când Ana este confundată cu Maria, când rolurile se inversează și cum crește eroarea cu numărul relațiilor suprapuse.

Compresia nu învinge aritmetica informației

O memorie cu b biți are cel mult 2^b stări distincte. Dacă vrem să distingem mai multe situații decât atât, trebuie să folosim structură suplimentară, să creștem memoria, să permitem pierderi sau să restrângem întrebările. O reprezentare cu numere reale de precizie ideală nu este o memorie fizică finită fără cost.

Acest argument nu respinge memoria distribuită. Spune doar că avantajul ei trebuie descris în termeni de redundanță exploatată, sarcini păstrate și erori acceptate. Aceeași cerință se aplică simbolurilor. Un model simbolic care folosește o regulă compactă poate câștiga enorm atunci când datele respectă regula; când nu o respectă, excepțiile pot ocupa mult spațiu.

O comparație onestă fixează bugetul de memorie și precizia numerică, apoi măsoară recuperarea și generalizarea. Nu compară o structură simbolică exactă care păstrează toate distincțiile cu un vector aproximativ fără a preciza pierderea tolerată.

Gradientul și căutarea discretă

Într-un model neuronal diferențiabil, parametrii pot fi ajustați folosind informație despre variația erorii. Această cale de optimizare este diferită de enumerarea unor circuite discrete. Dar optimizarea nu garantează automat atingerea soluției globale, iar existența unui gradient util depinde de reprezentare și de obiectiv.

SOP Lang ar putea folosi modele învățate pentru a propune operații, a recunoaște tipare și a sugera abstracții. Un verificator separat ar controla contractele relevante. O propunere neuronală greșită ar consuma buget, dar nu ar trebui să devină automat un fapt acceptat.

Există o limită inevitabilă: verificarea exactă nu este disponibilă pentru orice afirmație de limbaj natural. Dacă sarcina este rezumarea unui eseu ambiguu, nu avem întotdeauna o specificație completă. Sistemul trebuie atunci să combine verificări parțiale, surse și evaluări umane, marcând ce nu poate certifica.

Experimentul care ar decide ceva util

În loc să întrebăm ce paradigmă este în general superioară, putem fixa o sarcină: recuperarea rolurilor din evenimente compuse, răspunsul la întrebări noi și detectarea unei contradicții. Comparam o reprezentare relațională exactă, una vectorială și una hibridă. Păstrăm aceleași informații de intrare și măsurăm costul, rata de eroare și capacitatea de explicare.

O victorie a vectorilor la recuperare aproximativă ar justifica folosirea lor acolo. O victorie a relațiilor exacte la contradicții ar justifica un nucleu simbolic pentru acel rol. Scopul este o distribuire argumentată a responsabilităților, nu apărarea unei identități de școală.

18. Reprezentări suficiente pentru predicție și pentru întrebări viitoare

Statistica suficientă, printr-un exemplu familiar

Dacă vrem media unei liste, suma și numărul elementelor sunt suficiente. Nu avem nevoie de ordinea lor. Dacă vrem să știm dacă lista a fost crescătoare, aceleași două numere nu ajung. Suficiența este întotdeauna relativă la ceva ce dorim să calculăm sau să inferăm.

În statistică, o statistică suficientă păstrează informația relevantă despre un parametru în cadrul unui model specificat. Pentru cercetarea noastră, analogia este utilă, dar nu trebuie extinsă fără precizare: o interfață de circuit poate trebui să păstreze comportamentul pentru toate continuările admise, nu doar informație despre un parametru într-o distribuție fixată.

Există astfel trei noțiuni apropiate, dar distincte: suficiența exactă pentru un calcul, aproximarea sigură a posibilităților și păstrarea informației predictive într-un model probabilistic. Ele pot coopera, dar nu se substituie una alteia.

Stări predictive

Două istorii ale unui proces pot fi considerate echivalente dacă induc aceeași distribuție asupra viitoarelor observații, în condițiile modelului. Teoria mecanicii computaționale studiază asemenea stări predictive și relația lor cu reprezentarea structurii proceselor. [9]

Intuiția este apropiată de echivalența prin continuare: istorii diferite sunt reunite când nu mai schimbă predicțiile relevante. Dar egalitatea distribuțiilor nu este același lucru cu egalitatea unor valori observate pe câteva exemple. Estimarea ei din date finite introduce incertitudine și depinde de ipotezele despre proces.

Denumirea de „stări cauzale” folosită în această literatură nu trebuie confundată automat cu identificarea efectelor intervențiilor din modelele cauzale structurale. O stare suficientă pentru predicția observațiilor poate fi insuficientă pentru alegerea unei intervenții. Distincția din capitolul precedent rămâne necesară.

Information bottleneck

Metoda information bottleneck caută o reprezentare Z a datelor X care păstrează informația utilă despre o țintă Y și elimină detalii din X. Intuitiv, vrem un rezumat mic, dar încă predictiv. Un obiectiv clasic pune în balanță informația dintre X și Z și informația dintre Z și Y. [24]

Compromis conceptual: minimizăm $I(X; Z) - \beta I(Z; Y)$.

I descrie aici informația mutuală, adică dependența informațională dintre variabile în distribuția modelată. β stabilește cât preț punem pe predicția țintei față de comprimare. Formula nu este o garanție că orice semnificație importantă este păstrată. Ceea ce nu apare în țintă sau în distribuția de evaluare poate fi eliminat.

Varianta deterministic information bottleneck explorează un obiectiv bazat pe entropia reprezentării, favorizând o altă formă a comprimării. [25] „Determinist” se referă aici la reprezentare și la formularea metodei, nu la transformarea lumii într-un proces fără incertitudine și nici la demonstrarea automată a echivalenței semantice.

Schimbarea întrebării este testul decisiv

O sinteză despre un document poate păstra tema și concluzia generală, dar poate pierde o excepție rară. Pentru un cititor interesat de idee, comprimarea poate fi bună. Pentru un

cercetător care întreabă exact despre excepție, ea este insuficientă. Un singur scor mediu de rezumare poate ascunde această diferență.

Pentru SOP Lang, soluția propusă este separarea memoriei persistente de vederea adaptată întrebării. Sursa și reprezentarea mai bogată rămân accesibile. Pentru întrebarea curentă construim o interfață mai mică. Când întrebarea se schimbă, putem rafina sau reconstrui vederea, în loc să pretindem că vechiul rezumat este universal.

Această organizare costă memorie și administrare. Trebuie comparată cu alternativele: păstrarea integrală a tuturor derivărilor, păstrarea numai a textului sau comprimarea ireversibilă. Criteriul nu este o preferință abstractă pentru memorie bogată, ci raportul dintre cost și numărul întrebărilor viitoare la care sistemul poate răspunde corect.

O ipoteză despre înțelegere

Putem formula acum o ipoteză exploratorie: o parte a înțelegerii constă în descoperirea unor reprezentări care păstrează ceea ce rămâne invariant sub variații relevante și lasă deoparte detalii care nu schimbă consecințele urmărite. Aceasta explică de ce un expert poate vedea aceeași structură în probleme cu aparențe diferite.

Ipoteza nu spune că toate detaliile sunt zgomot sau că orice problemă are o esență mică. Uneori o excepție este chiar esența întrebării. Un sistem matur ar trebui să învețe și când o comprimare încetează să fie legitimă. Competența de a redeschide o distincție poate fi la fel de importantă ca abilitatea de a o elimina.

PARTEA A V-A

O arhitectură conceptuală pentru SOP Lang

O propunere de organizare a cunoașterii, a sintezei și a verificării; observații controlate și limite explicite.

19. Ce trebuie să poarte o interfață semantică

Mai mult decât o valoare, mai puțin decât întreaga istorie

Un fir semantic ar trebui să permită continuării să afle ce poate folosi fără să inspecteze din nou tot trecutul. Uneori aceasta înseamnă o valoare exactă. Alteori înseamnă o relație, un set de alternative sau un rezultat condiționat. Propunerea nu este ca fiecare fir să transporte obligatoriu aceeași structură enormă. Este să existe o disciplină comună a întrebărilor pe care interfața trebuie să le poată lămuri.

Prima întrebare privește natura conținutului: este o persoană, un eveniment, o relație, o interpretare candidată sau un circuit? A doua privește garanția: conținut exact în model, aproximare care include toate posibilitățile, ipoteză verificată numai pe exemple sau propunere euristică? A treia privește dependențele: sub ce premise și în ce context este valabil?

O parte a informației poate fi accesată prin referințe, nu copiată pe fiecare fir. Proveniența poate fi într-un graf comun. Schema poate fi declarată de tip. O relație poate fi stocată o singură dată și folosită de mai multe valori. Disciplina interfeței privește disponibilitatea și sensul informației, nu o obligație de duplicare fizică.

Trei niveluri de interfață

Nivelul operațional descrie ce poate consuma și produce componenta. Nivelul semantic precizează relația dintre intrare și ieșire. Nivelul epistemic spune de ce avem dreptul să folosim acea relație în cazul curent. O componentă poate fi perfect executabilă la primul nivel și insuficient justificată la al treilea.

De exemplu, un extractor primește text și produce un eveniment de transfer. Contractul operațional este clar. Contractul semantic ar trebui să spună că evenimentul corespunde unui anumit fragment. Statutul epistemic poate rămâne „interpretare candidată". O regulă ulterioară de posesie poate fi exactă față de eveniment, dar răspunsul final moștenește incertitudinea interpretării.

Această organizare împiedică o eroare comună: transformarea unei presupuneri în certitudine prin trecerea ei prin multe operații exacte. Zece pași corecți după o premisă nesigură nu fac premisa sigură. Un circuit trebuie să păstreze dependența, nu să spele incertitudinea prin calcul.

Biblioteca și interpreții

O bibliotecă de circuite ar trebui să expună ce probleme poate rezolva, în ce condiții și ce reprezentări folosește. Expanderul poate consulta aceste contracte pentru a găsi operații compatibile cu obiectivul. O operație compusă poate fi extinsă în subcircuit când este necesară analiza detaliată.

Interpreții realizează operațiile într-o semantică aleasă. Un interpret poate calcula exact aritmetică finită. Altul poate evalua constrângeri. Altul poate propune o interpretare lingvistică. Uniformitatea apelului nu implică uniformitatea garanțiilor. Sistemul trebuie să poată inspecta diferența.

O întrebare importantă privește baza de încredere: care operații și verificatoare sunt presupuse corecte pentru ca restul explicației să funcționeze? O bază mică poate fi testată și analizată mai bine, dar nu dispare prin metaprogramare. Un circuit care verifică alt circuit se bazează tot pe semantica operațiilor sale. Reprezentarea uniformă nu înlătură această ancoră.

Expanderul nu trebuie să aleagă imediat un singur program

Propunerea principală este ca expanderul să construiască un graf partajat al continuărilor posibile. Unele noduri reprezintă obligații încă nerezolvate; altele, rezultate intermediare; altele, clase de construcții echivalente. Legăturile arată compuneri, alternative sau justificări, fără a confunda aceste sensuri.

Obiectivul generează cerințe. Informația disponibilă generează posibilități. Contractele operațiilor permit întâlnirea lor. Reprezentările abstracte ieftine elimină candidați incompatibili. Reprezentările exacte sau verificatoarele mai puternice controlează acceptarea. Când interfața este prea săracă, ea este rafinată sau cazul este separat în alternative.

Acest proces poate fi determinist în alegerea și verificarea pașilor, poate folosi euristici învățate sau le poate combina. Determinismul este o proprietate a procedurii, nu o garanție de cost mic. O procedură complet deterministă poate explora exponențial de multe cazuri.

Ce înseamnă progres

Nu orice pas bun micșorează imediat mulțimea posibilităților. Descoperirea unei ambiguități poate crește numărul interpretărilor reprezentate. Adăugarea unei relații poate

mări starea și reduce căutarea ulterioară. Introducerea unei ipoteze poate crea ramuri noi, dar poate permite o explicație care lipsea.

De aceea, un singur indicator precum „incertitudinea totală scade la fiecare pas” ar fi prea rigid. Putem măsura separat obligațiile rezolvate, distincțiile verificate, candidații eliminați sigur, costul rămas estimat și calitatea rezultatului. Când sistemul redeschide o decizie greșită, aceasta poate fi progres chiar dacă graful devine temporar mai mare.

O mașină de înțelegere trebuie să poată spune nu doar „am găsit un circuit”, ci și „acest circuit satisface obiectivul în aceste ipoteze; acestea sunt alternativele încă relevante; aici se află limita verificării”. Aceasta este forma de rezultat pe care arhitectura o urmărește.

20. Anatomia unei întrebări rezolvate fără a inventa ce lipsește

Situația

Vom urmări un exemplu conceptual complet. Textul spune: „Luni dimineață, Ana avea cheia laboratorului. La prânz i-a dat cheia Mariei. Ea a plecat la bibliotecă. Marți dimineață, paznicul a găsit cheia pe masa laboratorului.” Întrebarea inițială este: „Cine avea cheia imediat după transferul de luni?”

Acceptăm pentru acest exemplu o semantică restrânsă: „a da cheia” descrie un transfer fizic încheiat; cheia menționată este aceeași; „imediat după” nu include un eveniment ulterior nementionat în același moment. Acestea sunt premisele modelului, nu adevăruri despre orice utilizare a cuvintelor. Sistemul trebuie să le poată afișa sau să le lege de regulile domeniului.

Nu vom pretinde că un prototip a parcurs deja toate etapele de mai jos. Exemplul descrie comportamentul pe care programul de cercetare dorește să îl obțină și să îl testeze.

Interpretările inițiale

Ingestia identifică persoanele, obiectul, locurile și evenimentele. Construiește o interpretare pentru transferul către Maria și două alternative pentru pronumele „ea”: Ana sau Maria. Evenimentul de marți este reprezentat separat, după toate evenimentele de luni.

În loc să copieze întreaga situație în două lumi independente, sistemul păstrează structura comună și o alegere locală pentru agentul plecării. Consecutiv, orice concluzie care depinde de acea alegere poartă condiția potrivită. Un fapt care nu depinde de ea poate fi calculat o singură dată.

Întrebarea indică un obiect, un moment și o relație cerută: posesia cheii imediat după transfer. Expanderul caută operații care pot produce acea relație. Regula de transfer este candidată fiindcă are o ieșire de forma potrivită și condiții compatibile cu informația disponibilă.

Propagarea cerințelor

Regula cere un destinatar identificat, un obiect și un eveniment actual, nu ipotetic. Aceste condiții sunt satisfăcute în modelul acceptat. Mai cere plasarea întrebării imediat după transfer. Relația temporală este dată chiar de formularea întrebării.

Agentul plecării la bibliotecă nu apare în cerințele necesare. Sistemul poate demonstra, pentru această bibliotecă de reguli, că ambele interpretări ale pronumelui conduc la aceeași posesie imediat după transfer. Astfel, nu are nevoie să

aleagă cine a plecat. Aceasta nu este ignorarea arbitrară a unei ambiguități, ci demonstrarea irelevanței ei pentru observația curentă.

Evenimentul de marți nu schimbă retrospectiv răspunsul. Poate furniza informație despre o stare ulterioară, dar nu dovedește cine a adus cheia înapoi. Circuitul nu trebuie să inventeze un traseu complet doar pentru a lega narativ începutul de sfârșit.

Răspunsul și justificarea

Răspunsul conceptual este: Maria avea cheia imediat după transfer, în interpretarea literală de transfer fizic. Justificarea indică evenimentul și regula folosită. Poate menționa că identitatea persoanei care a plecat la bibliotecă nu este necesară concluziei.

Un certificat formal restrâns ar verifica legăturile dintre eveniment, destinatar, obiect și moment. O verificare a fidelității față de text ar examina dacă „Mariei" este într-adevăr destinatarul și dacă evenimentul nu este condițional. Acestea sunt două verificări diferite. A doua poate necesita un evaluator lingvistic sau un om, în funcție de domeniul controlat.

Circuitul final poate fi mic. Graful de construcție poate păstra alternativele și motivele pentru care unele nu au fost explorate. Pentru auditul obișnuit, afișăm justificarea scurtă. Pentru investigarea unui eșec, trebuie să putem deschide și traseul de construcție relevant.

Schimbarea întrebării

Întrebăm acum: „Cine a plecat la bibliotecă?” Vechea interfață pentru posesie este insuficientă. Sistemul revine la reprezentarea mai bogată și constată două interpretări. Răspunsul corect este ambiguitatea, dacă nu există context suplimentar.

Întrebăm apoi: „Cine a adus cheia înapoi în laborator?” Textul nu precizează. Faptul că paznicul o găsește acolo marți nu identifică persoana care a transportat-o. O componentă generativă poate propune scenarii, dar acestea trebuie separate de răspunsul derivat. „Nu rezultă din text” este o concluzie utilă despre limitele informației.

Întrebarea „ar fi putut Maria să intre în laborator luni seara?” cere reguli suplimentare: dacă avea cheia la acel moment, dacă cheia deschide laboratorul, dacă accesul era permis și dacă posedarea cheii este suficientă. Un sistem care răspunde doar fiindcă recunoaște asocierea dintre chei și uși poate sări peste condiții decisive.

Corectarea unei premise

Adăugăm: „Corecție: Ana i-a arătat cheia Mariei, dar nu i-a dat-o.” Nu avem o rafinare monotonă a vechiului transfer; retragem interpretarea acelui eveniment. Concluzia despre posesia Mariei își pierde suportul. Informațiile independente, precum existența persoanelor sau găsirea cheii marți, pot rămâne.

Acesta este un test puternic pentru arhitectură. Dacă sistemul păstrează răspunsul vechi fiindcă apare în memorie, proveniența nu controlează acceptarea. Dacă reconstruiește întregul document fără să poată identifica dependența afectată, poate fi corect, dar nu a demonstrat avantajul revizuirii locale.

Ce am învățat din exemplu

Înțelegerea operațională nu a cerut alegerea tuturor detaliilor unei lumi complete. A cerut o reprezentare suficientă pentru întrebarea curentă, păstrarea accesului la distincțiile nerezolvate și disciplina de a nu completa golurile ca fapte. Același exemplu ar putea deveni o familie de teste prin schimbarea participanților, a timpului, a verbului și a întrebării.

Criteriul de succes nu este că sistemul spune mereu „Maria”. Este că răspunsul se schimbă exact când schimbarea textului sau a întrebării o justifică și că explicația indică acea schimbare. Aceasta face diferența dintre memorarea unui tipar și folosirea unei structuri.

21. Cum ar putea fi învățate circuitele și interfețele lor

Trei obiecte diferite ale învățării

Un sistem poate învăța parametri într-o operație fixată. Poate învăța un circuit nou prin compunerea operațiilor existente. Poate învăța o reprezentare nouă în care multe circuite devin mai ușor de construit. Aceste trei forme de învățare nu trebuie raportate sub aceeași etichetă fără diferențiere.

Pentru SOP Lang, prima poate ajuta recunoașterea limbajului. A doua poate produce reguli și proceduri reutilizabile. A treia este legată direct de interfețele suficiente. Ambiția este ca sistemul să descopere că anumite detalii ale istoriei nu mai contează și că alte relații trebuie păstrate obligatoriu.

O cale prudentă începe cu interfețe furnizate de cercetător, apoi cere sistemului să aleagă între ele. Următoarea etapă permite combinarea lor. Numai după aceea cerem inventa-

rea unor interfețe noi. Altfel, un eșec total nu spune dacă problema este selecția, compunerea sau descoperirea reprezentării.

Biblioteci care cresc din soluții

Dacă mai multe probleme folosesc același subcircuit, îl putem transforma într-o abstracție reutilizabilă. În loc să căutăm de fiecare dată o succesiune lungă de operații, alegem un singur concept de nivel mai înalt. DreamCoder explorează învățarea de biblioteci simbolice împreună cu ghidarea neuronală a căutării. [23]

Reutilizarea poate reduce adâncimea căutării, dar creșterea bibliotecii mărește numărul opțiunilor. O bibliotecă cu mii de operații aproape identice poate fi mai greu de căutat decât una mică. Trebuie învățate și organizarea, contractele și criteriile de selecție.

O abstracție utilă trebuie să comprime mai mult decât un singur exemplu. Dacă denumim fiecare problemă rezolvată ca operație nouă, am construit o memorie de cazuri. Aceasta poate fi folositoare, dar nu dovedește descoperirea unei regularități generale. Testul este reutilizarea pe combinații și instanțe care nu au fost prezente la construire.

Lungimea descrierii și prețul excepțiilor

Principiul minimum description length sugerează evaluarea împreună a complexității modelului și a costului descrierii datelor rămase după folosirea modelului. [26] În termeni de bibliotecă, putem pune în balanță costul noilor circuite și costul soluțiilor exprimate cu ajutorul lor.

O regulă care comprimă o mie de cazuri și cere două excepții poate fi valoroasă. O regulă care cere câte o excepție pentru fiecare caz nou nu a descoperit multă structură. Totuși,

descrierea cea mai scurtă nu este automat explicația adevărată. Alegerea limbajului, a codării și a obiectivului influențează rezultatul.

Pentru cercetare, MDL poate fi un criteriu de selecție, nu un substitut al validării semantice. O bibliotecă foarte compactă poate șterge distincții rare, dar esențiale. Trebuie evaluată pe întrebări care le cer, nu doar pe frecvența medie a cazurilor.

Învățarea unei relații de reunire

Sistemul poate observa că două construcții au același comportament pe probele curente. Propune un candidat la interfață comună. Un verificator încearcă să găsească o continuare care le deosebește. Dacă o găsește, interfața se îmbogățește. Dacă există o teorie decidabilă pentru clasa respectivă, putem obține și o demonstrație. În alte cazuri rămânem la o garanție empirică delimitată.

Cercetarea asupra inferării regulilor de rescriere folosind equality saturation este un precedent pentru descoperirea unor egalități utile în spații de expresii. [31] Transferul către semantica documentelor cere însă să știm ce sunt obiectele comparate și ce verificator poate controla sensul lor.

Nu este suficient să învățăm un vector în care două stări sunt apropiate. Apropierea poate ghida căutarea unei echivalențe. Acceptarea reunificării exacte cere un argument suplimentar. Într-un nivel aproximativ, putem permite erori, dar acestea trebuie izolate de nivelul care promite concluzii verificate.

Învață sistemul sau cercetătorul?

O evaluare transparentă raportează cine a furnizat primitivele, schema semantică, exemplele, regulile și criteriul de oprire. Dacă un model lingvistic extern produce circuitul

complet, iar SOP Lang îl execută, rezultatul poate demonstra interoperabilitate și audit. Nu demonstrează încă sinteză autonomă în nucleul simbolic.

Dacă cercetătorul inspectează fiecare eșec și adaugă manual regula potrivită, progresul este un proces de inginerie asistată. Aceasta nu este o rușine metodologică. Devine problematic numai când munca este ascunsă și rezultatul este prezentat ca învățare independentă.

Etapa matură ar cere ca sistemul să propună îmbunătățiri, să le verifice într-un cadru declarat și să le accepte numai când reduc costul total sau cresc competența fără pierderi necontrolate. Până acolo, fiecare nivel intermediar poate produce rezultate științifice utile.

22. Auditabilitatea: de la urmă completă la explicație folosibilă

O urmă nu este încă o explicație

Un jurnal cu un milion de operații poate permite reconstituirea execuției și totuși să nu ajute un cercetător să înțeleagă eroarea. Auditabilitatea practică cere selecția structurii relevante: premisele decisive, transformările care schimbă sensul și locul în care s-a introdus o aproximare.

Pentru SOP Lang, propunem trei vederi asupra aceluiași rezultat. Vederea scurtă arată justificarea principală. Vederea de diagnostic arată alternativele și verificările care au determinat alegerea. Vederea de reproducere permite refacerea calculului relevant. Acestea sunt moduri de prezentare ale unei urme, nu motive pentru a inventa o poveste diferită de calculul efectiv.

Un sistem poate produce o explicație pedagogică mai simplă decât traseul real, dar trebuie să o eticheteze ca explicație reconstruită și să arate legătura verificabilă cu rezultatul. Nu trebuie să pretindă că povestea simplificată este descrierea literală a tuturor pașilor interni.

Proveniența valorilor și proveniența deciziilor

Proveniența valorii spune de unde vine o concluzie: surse, reguli, calcule. Proveniența deciziei de căutare spune de ce a fost încercat un circuit înaintea altuia. Prima poate fi suficientă pentru verificarea rezultatului. A doua este importantă pentru înțelegerea costurilor, a erorilor de selecție și a posibilelor preferințe sistematice.

O euristică neuronală poate ordona candidații fără să poată explica intuitiv scorurile. Dacă verificarea ulterioară este solidă, aceasta nu compromite automat validitatea formală a rezultatului. Dar poate afecta ce soluții sunt găsite în buget și ce cazuri sunt abandonate. Auditul performanței trebuie să includă această selecție.

Un rezultat „nu am găsit” poate însemna imposibilitate demonstrată, epuizarea unei căutări finite complete sau terminarea bugetului. Aceste trei situații trebuie prezentate diferit. Un verificator care acceptă soluții nu este automat capabil să certifice că nicio soluție nu există.

Revizuire și invalidare locală

Dacă o sursă este retrasă, sistemul ar trebui să identifice concluziile care depind de ea și să verifice dacă au suport alternativ. Un graf de proveniență partajat poate permite această operație fără reanalizarea tuturor rezultatelor. Dar reutilizarea sigură cere ca dependențele să fie complete pentru proprietățile urmărite.

Cache-urile trebuie indexate după contextul relevant. Două întrebări cu același text, dar cu reguli de domeniu schimbate, nu au neapărat același răspuns. O interfață care ignoră versiunea unei ipoteze poate reutiliza o concluzie invalidată. Acesta este un caz de insuficiență semantică, nu doar o eroare administrativă.

Pentru sisteme care acționează, planul și efectul executat trebuie separate. O operație poate propune trimiterea unui mesaj fără să îl trimită. Verificarea unui plan nu este autorizarea tuturor efectelor. Aceste distincții nu domină teoria cărții, dar devin necesare când circuitele ies din calculul pur.

Cum măsurăm auditabilitatea

Putem construi sarcini de diagnostic cu o premisă greșită, o regulă greșită sau o reunire nejustificată. Cercetători care nu au construit sistemul primesc explicațiile și trebuie să localizeze problema. Măsurăm timpul, proporția diagnosticelor corecte și numărul de detalii pe care trebuie să le consulte.

Putem compara explicația scurtă cu jurnalul brut și cu un sistem fără proveniență. Dacă structura explicită nu reduce costul diagnosticului, promisiunea de inteligibilitate rămâne nedemonstrată. Dacă îl reduce numai pentru autorii sistemului, trebuie îmbunătățite interfețele pentru utilizatori externi.

Auditabilitatea poate intra în conflict cu confidențialitatea sau cu volumul datelor. O explicație poate dezvălui informații sensibile chiar când răspunsul final este public. Proiectul trebuie să precizeze ce poate vedea fiecare evaluator și ce garanții pot fi verificate prin certificate mai restrânse. Transparent nu trebuie să însemne nediferențiat accesibil tuturor.

23. Ce indică modelele controlate și ce nu indică

De ce avem nevoie de modele mici

Un model mic poate izola mecanismul pe care îl susținem. Dacă vrem să știm dacă reunirea semantică reduce căutarea, nu avem nevoie inițial de toate dificultățile limbajului natural. Avem nevoie de o familie în care echivalența este exactă și în care putem compara rezultatul cu o metodă independentă.

Observațiile prezentate aici provin din asemenea modele finite, reexecutate și verificate. Ele justifică interesul pentru mecanismele testate. Nu constituie o demonstrație a unei mașini generale de înțelegere și nu validează toate componentele arhitecturii propuse.

Reunirea exactă a unor construcții funcționale

Într-o familie de circuite aritmetice finite, fiecare etapă permite două transformări. Cu n etape există 2^n secvențe de alegeri. Totuși, transformările păstrează o formă afină a funcției, f(x) = ax + b, cu aritmetică modulo 31. Coeficientul a rămâne nenul, deci există cel mult 30 × 31 = 930 de funcții de această formă.

Perechea coeficienților, împreună cu etapa, permite reunirea exactă a construcțiilor care au același comportament funcțional. La 16 etape, cele 65 536 de secvențe posibile au fost reprezentate prin 430 de funcții finale distincte; explorarea înainte a evaluat 4 794 de extinderi. La 40 de etape, numărul secvențelor posibile este, prin formulă, 1 099 511 627 776, iar explorarea înainte a evaluat 41 110 extinderi și a ajuns la 930 de funcții finale distincte.

La 80 de etape, numărul secvențelor posibile este aproximativ 1,21 × 10^24, iar numărul extinderilor evaluate înainte a fost 115 510. Acest contrast arată diferența dintre numărul istoriilor și numărul stărilor funcționale relevante. Valorile enorme ale numărului de secvențe sunt calculate prin formula combinatorială; ele nu au fost obținute prin enumerarea efectivă a tuturor secvențelor.

Ce a fost verificat independent

Pentru dimensiuni de până la 16 etape, un enumerator independent a verificat funcțiile și multiplicitatea construcțiilor care le produc. Pentru cazurile mai mari au fost verificate reprezentările finale pe toate cele 31 de intrări ale domeniului și a fost reconstruit un circuit care satisface funcția țintă. Nu s-a folosit doar un exemplu de intrare pentru a declara egalitatea funcțiilor.

Numerele de mai sus privesc extinderile înainte, nu costul total al întregii proceduri. Au existat și verificări înapoi, verificări ale funcțiilor, stocare și operații de numărare. Nu prezentăm aceste valori ca măsurători de timp sau ca factori de accelerare față de un competitor optimizat.

Reprezentarea afină a fost furnizată, nu descoperită. Modulul 31 a fost mic și fixat. Dacă modulul devine m, numărul stărilor poate crește de ordinul lui m^2; aceasta nu înseamnă automat cost polinomial în numărul de biți necesari pentru a scrie m. Nu s-au testat topologii arbitrare, interpretare lingvistică sau învățare autonomă a interfeței.

Un contraexemplu la reunirea după prea puține probe

În același tip de domeniu, două funcții pot coincide pe intrarea zero și pot diferi pe intrarea unu. Acest control simplu confirmă că un răspuns comun pe un exemplu nu justifică

reunirea universală. O semnătură empirică este cel mult un filtru pentru candidați, dacă nu există verificare suplimentară.

Rezultatul pozitiv și controlul negativ trebuie citite împreună. Nu este suficient că o reprezentare comprimă. Trebuie să arătăm și că nu comprimă două stări care ar trebui distinse. O compresie perfectă care identifică toate circuitele într-o singură stare ar avea dimensiune excelentă și utilitate semantică aproape nulă.

Relațiile detectează un conflict pe care domeniile individuale îl lasă nedecis

Au fost analizate și sisteme binare de egalități și inegalități exprimate prin adunare modulo doi. În cicluri cu o paritate finală incompatibilă, verificarea suporturilor locale ale valorilor nu elimina nicio valoare individuală. Rezultatul corect al acelei proceduri era „nedecis”. Păstrarea relațiilor și eliminarea liniară produceau un certificat explicit de contradicție.

Au fost verificate cicluri consistente și inconsistente până la 256 de variabile. Cazurile mici, până la 16 variabile, au fost controlate și prin enumerare completă. În cazurile consistente de această formă existau două soluții, iar în cele inconsistente combinația ecuațiilor producea 0 = 1.

Aceasta nu demonstrează că orice propagare locală eșuează sau că numai o metodă globală sofisticată poate rezolva problema. Ciclurile au structură simplă; metode care păstrează legături de paritate le pot rezolva eficient. Rezultatul izolează slăbiciunea reprezentării exclusiv individuale și valoarea unei algebre relaționale potrivite.

Interpretarea comună a observațiilor

Cele două familii de rezultate susțin aceeași direcție din unghiuri opuse. În prima, un rezumat exact reunește istorii inutile și reduce căutarea. În a doua, un rezumat prea sărac pierde dependența care ar permite concluzia. Nu există contradicție: trebuie uitat ceea ce nu mai contează și păstrat ceea ce continuarea poate folosi.

Ce rămâne de demonstrat este mult mai ambițios: că asemenea reprezentări pot fi găsite pentru sarcini utile de limbaj, că pot fi învățate, că verificarea lor rămâne economică și că avantajul persistă când variază domeniul, întrebările și sursele. Modelele finite justifică explorarea acestor întrebări; nu furnizează deja răspunsul lor.

PARTEA A VI-A

Programul de cercetare

Ipoteze testabile, familii de experimente, criterii de infirmare și etape pentru cercetători viitori.

24. Ipoteze care pot fi infirmate

De la o viziune la afirmații riscante

„Circuitele vor face inteligența mai clară” este o aspirație prea largă pentru a fi testată direct. Trebuie descompusă în afirmații care pot eșua separat. Un rezultat negativ despre

una nu trebuie să fie ascuns prin succesul alteia. Viteza, corectitudinea, generalizarea și auditabilitatea sunt obiective diferite.

Vom considera o ipoteză utilă atunci când precizează domeniul, mecanismul, comparația și observația care ar pune-o în dificultate. Nu cerem ca fiecare ipoteză să aibă o singură cifră universală. Cerem ca pragurile și măsurătorile să fie stabilite înainte de alegerea exemplelor favorabile.

H1 — Reunirea semantică reduce căutarea fără a schimba soluțiile

Într-o familie finită de probleme, o interfață declarată exactă ar trebui să păstreze mulțimea răspunsurilor obținute de enumerarea completă și să reducă numărul extinderilor evaluate atunci când există multe istorii echivalente. Baseline-ul nu trebuie să fie intenționat slab: includem și memorarea după identitate sintactică, nu doar enumerarea naivă.

Un singur caz în care un răspuns valid dispare sau unul invalid apare infirmă exactitatea implementării ori a interfeței pentru clasa declarată. O lipsă a câștigului de cost nu infirmă exactitatea, dar infirmă avantajul practic în regimul testat. Cele două rezultate se raportează separat.

Fenomenul așteptat este saturarea numărului de stări distincte atunci când spațiul semantic accesibil este mic, în timp ce numărul istoriilor continuă să crească. Dacă numărul stărilor urmărește aproape numărul istoriilor, nu există suficientă reutilizare pentru mecanismul propus.

H2 — Relațiile potrivite elimină blocaje ale rezumatelor individuale

Comparăm aceleași probleme cu domenii independente pe variabile și cu reprezentări care păstrează o clasă de relații. Ipoteza nu este că relațiile sunt întotdeauna mai ieftine. Este

că, în familiile în care dependența omisă este decisivă, costul suplimentar al relației poate evita ramificare mult mai scumpă.

Măsurăm conflictele detectate, cazurile rămase nedecise, memoria și costul total. Un rezultat negativ interesant apare când domeniul relațional crește atât de mult încât avantajul dispare. Acesta sugerează folosirea selectivă a relațiilor sau alegerea unei alte algebre, nu eliminarea neapărată a întregii direcții.

H3 — Ambiguitatea poate fi păstrată selectiv, fără alegeri premature

Pe texte cu interpretări controlate, sistemul ar trebui să răspundă corect întrebărilor invariabile între interpretări și să păstreze nedeterminarea când răspunsurile diferă. Comparația include un sistem care alege imediat cea mai probabilă interpretare și unul care enumeră toate interpretările complete.

Succesul înseamnă reducerea costului față de enumerare, fără creșterea concluziilor nesusținute, și mai puține erori decât alegerea prematură pe cazurile ambigue. Dacă sistemul răspunde aparent bine doar fiindcă setul de întrebări evită toate ambiguitățile relevante, testul nu susține ipoteza generală.

H4 — Interfețele pot fi învățate și reutilizate

Interfețele descoperite pe o familie de probleme ar trebui să reducă costul pe instanțe noi sau pe compuneri noi, fără a cere o excepție pentru fiecare caz. Comparația include interfețe furnizate de un expert, interfețe generice și absența comprimării semantice.

Costul învățării, numărul de contraexemple și munca de validare trebuie incluse. O interfață care economisește o secundă la o singură interogare după o săptămână de căutare poate fi un rezultat teoretic interesant, dar nu un avantaj practic în acel regim. Pentru mii de interogări, aceeași investiție poate avea altă valoare.

H5 — Verificarea și selecția pot fi separate util

O strategie învățată ar trebui să găsească mai repede candidați buni, în timp ce verificarea păstrează garanția declarată. Comparăm ordonări deterministe simple, euristici construite manual și propuneri învățate, folosind același verificator și același buget.

Dacă acuratețea pare mai bună numai fiindcă verificarea este mai permisivă, ipoteza nu este confirmată. Dacă strategia învățată ratează sistematic soluții rare în buget, efectul trebuie raportat chiar când media crește. Politica de explorare poate introduce o preferință structurală pentru cazurile frecvente.

H6 — Urmele structurale reduc costul diagnosticului

Explicațiile circuitale ar trebui să ajute evaluatori independenți să localizeze o eroare mai repede și mai corect decât explicațiile fără proveniență sau jurnalele brute. Măsurăm timpul și calitatea diagnosticului, nu doar satisfacția declarată față de text.

Un eșec aici ar fi important: sistemul poate fi verificabil formal și totuși greu de înțeles pentru oameni. Ar trebui atunci cercetată prezentarea explicațiilor, fără a susține că simpla existență a circuitului rezolvă inteligibilitatea.

O ipoteză filosofică rămasă deschisă

Ipoteza mai largă este că descoperirea distincțiilor relevante reprezintă o componentă centrală a înțelegerii. Experimentele pot susține această idee prin convergența rezultatelor, dar nu o pot transforma imediat într-o definiție exhaustivă a minții. Competențe sociale, scopuri, corp, experiență și context pot cere dimensiuni suplimentare.

Valoarea programului nu depinde de monopolizarea cuvântului „înțelegere". Dacă obținem sisteme care generalizează mai bine, greșesc mai vizibil și pot fi corectate mai precis, rezultatul este important chiar într-o interpretare filosofică prudentă.

25. Laboratorul formal: ce simulări ar trebui construite

Familia A — Stări semantice mici, istorii foarte multe

Începem cu transformări care păstrează o familie cunoscută: funcții afine finite, permutări ale unui domeniu mic, automate pentru proprietăți ale șirurilor sau operații pe mulțimi finite. Variem separat lungimea construcției, dimensiunea domeniului și numărul operațiilor disponibile.

Pentru fiecare familie, cercetătorul definește interfața exactă și verifică proprietatea de compunere. Pe cazuri mici, un enumerator independent stabilește adevărul de referință. Pe cazuri mai mari, verificăm martori, invarianta interfeței și, unde este posibil, certificate formale. Nu folosim același mecanism pentru a produce și a confirma toate răspunsurile.

Căutăm trei regimuri: comprimare puternică, avantaj moderat și aproape nicio comprimare. Toate trebuie publicate. Un rezultat care arată doar regimul favorabil nu ne spune unde se poate aplica metoda.

Familia B — Reprezentări prea sărace, potrivite și prea bogate

Luăm aceeași clasă de probleme și schimbăm numai interfața. O variantă păstrează valori individuale. Alta adaugă egalități. Alta păstrează ecuații liniare sau relații mai generale. O variantă extremă păstrează întreaga stare concretă.

Fenomenul căutat este un compromis: interfața prea săracă produce multe cazuri nedecise sau, dacă este folosită greșit ca exactă, erori; interfața potrivită reduce căutarea; interfața prea bogată poate consuma costuri mari de stocare și normalizare. Nu presupunem anticipat că există un singur minim sau că el este stabil între familii.

Un rezultat util ar fi o regulă de selecție adaptivă: pornește cu o reprezentare ieftină și adaugă o relație când un contraexemplu arată că este necesară. Trebuie comparată cu alegerea de la început a unei reprezentări bogate, incluzând costul tuturor rafinărilor intermediare.

Familia C — Topologia și algebra sunt variate independent

Generăm grafuri de constrângeri cu conectivitate mică sau mare și relații cu sau fără structură algebrică exploatabilă. Un ciclu de relații binare simple nu reprezintă același tip de dificultate ca un graf dens de constrângeri arbitrare. O matrice liniară densă poate fi mai ușor de rezolvat decât o familie rară de relații neliniare.

Experimentul trebuie să evite confuzia dintre mărimea grafului și dificultatea semantică. Putem păstra numărul variabilelor și schimba relațiile, apoi păstra algebra și schimba topologia. Măsurăm dimensiunea frontierelor, costul eliminării, numărul ramificărilor și memoria maximă.

Un fenomen promițător ar fi recunoașterea automată a unui subcircuit liniar sau a unei independențe condiționate, urmată de utilizarea unui interpret specializat. Acesta ar ilustra cum cunoașterea structurii schimbă algoritmul, nu doar ordinea încercărilor.

Familia D — Echivalențe învățate și contraexemple adversariale

Pornim de la semnături empirice construite pe puține probe. Cerem sistemului să găsească stări pe care semnătura le reunește, dar pe care o continuare le distinge. Apoi observăm dacă rafinarea se stabilizează într-o reprezentare mică sau continuă să acumuleze excepții.

Contraexemplele trebuie căutate deliberat, nu așteptate întâmplător. Putem varia valorile-limită, ordinea operațiilor, domeniul numeric și precondițiile. Pentru limbaje cu operații parțiale, testăm și intrări neadmise. O rescriere corectă pentru numere ideale poate eșua într-o semantică cu depășiri sau valori speciale.

Raportăm separat egalitățile demonstrate, egalitățile verificate exhaustiv pe domenii finite și echivalențele susținute numai empiric. Aceste categorii pot exista simultan într-o bibliotecă, dar nu trebuie să împartă aceeași etichetă de certitudine.

Familia E — Memorie, recompunere și cost amortizat

O interfață poate avea valoare prin reutilizare. Construim secvențe de întrebări asupra aceleiași baze, apoi introducem modificări locale ale datelor sau regulilor. Comparăm recompilarea completă, actualizarea prin proveniență și reconstruirea vederilor din sursa persistentă.

Costul total poate fi scris conceptual ca suma costului de învățare, de compilare, al interogărilor, al verificărilor și al actualizărilor. Raportarea numai a latenței unei interogări deja pregătite nu este suficientă. Nici raportarea numai a costului inițial nu surprinde valoarea unei biblioteci reutilizate.

Un rezultat interesant ar fi existența unui prag de reutilizare după care interfața învățată devine economică. Altul ar fi constatarea că schimbările frecvente distrug avantajul compilării. Aceste rezultate ar ghida alegerea aplicațiilor potrivite.

Protocolul minim

Dimensiunile și distribuțiile problemelor trebuie precizate înaintea interpretării rezultatelor. Folosim mai multe instanțe și, când generarea sau selecția este aleatorie, mai multe inițializări. Raportăm dispersia, nu numai media. Pentru proceduri deterministe, variem familiile și ordinea problemelor, deoarece determinismul nu elimină sensibilitatea la intrare.

Păstrăm atât cazurile rezolvate, cât și eșecurile, expirările bugetului și consumurile excesive. Un rezultat corect obținut după un cost enorm nu este același lucru cu un rezultat fezabil. Un rezultat rapid, dar incorect, nu trebuie ascuns într-o medie de viteză.

26. Laboratorul limbajului: de la texte controlate la documente reale

De ce începem cu un limbaj controlat

Pentru a evalua raționamentul, trebuie să știm dacă interpretarea intrării este corectă. În texte naturale complet deschise, eșecul poate proveni din vocabular, pragmatică, cu-

noaștere absentă sau sinteză. Un limbaj controlat permite izolarea acestor componente fără a pretinde că este deja întregul limbaj natural.

În prima etapă, cercetătorul generează situații structurate și texte corespunzătoare. Situația oferă adevărul de referință. Testele includ parafraze, nu doar un singur șablon. Mai târziu, textele umane sunt adnotate independent, cu marcarea dezacordurilor legitime.

Un test bine organizat separă trei condiții: raționament pe reprezentări corecte furnizate direct, interpretare evaluată separat și sistem complet de la text la răspuns. Diferența dintre rezultate arată unde se află dificultatea. Nu trebuie să atribuim parserului o eroare a planificării sau invers.

Un curriculum al fenomenelor

Începem cu identitate, apartenență și transferuri simple. Adăugăm apoi ordinea temporală și persistența. Urmează coreferința, alternativele, negația explicită, cuantificatorii și condiționalele. Abia după ce acestea sunt controlate introducem rapoarte despre credințe, explicații cauzale și cunoaștere de fundal variabilă.

Pentru fiecare fenomen există perechi minimale de texte: schimbăm un singur element care ar trebui să schimbe răspunsul. „Toți” devine „unii”; „înainte” devine „după”; „a dat” devine „a arătat”; „spune că” este adăugat înaintea unei afirmații. Mai există și schimbări care nu ar trebui să afecteze răspunsul: redenumirea consecventă a persoanelor sau adăugarea unui detaliu independent.

Evaluarea trebuie să includă compuneri noi ale fenomenelor, nu doar exemple noi din aceleași șabloane. Un sistem care stăpânește separat negația și timpul poate eșua când negația privește un eveniment dintr-un interval. Generalizarea compozițională trebuie testată explicit.

Întrebări care cer reținerea distincțiilor rare

Setul de teste trebuie să conțină întrebări despre excepții, nu numai despre concluzia dominantă. O reprezentare care netezește diferențele rare poate produce rezumate plăcute și răspunsuri greșite exact în cazurile importante. Aceasta este o problemă atât pentru modele statistice, cât și pentru reguli simbolice prea generale.

Putem construi documente în care nouă evenimente respectă o regulă, iar al zecelea introduce o excepție explicită. Întrebăm mai întâi despre tendință, apoi despre excepție. Un sistem suficient pentru prima întrebare nu trebuie să pretindă că este suficient și pentru a doua fără revenire la informația păstrată.

Măsurăm astfel costul schimbării obiectivului: câte detalii trebuie recuperate, cât durează rafinarea și dacă răspunsul vechi este retras când nu mai corespunde întrebării. Acesta este un test direct al separării dintre memoria persistentă și vederea specifică sarcinii.

Dovezi și benchmarkuri

ProofWriter și PRover sunt repere pentru sarcini de inferență și generare de dovezi din reguli exprimate lingvistic. Ele arată utilitatea evaluării împreună a concluziei și a justificării, în domenii delimitate. [28, 29] Nu trebuie tratate ca probe exhaustive ale înțelegerii deschise.

Pentru SOP Lang, putem adapta această idee fără să reducem totul la un singur scor de benchmark. Evaluăm corectitudinea răspunsului, validitatea pașilor, fidelitatea față de surse, capacitatea de abținere și reacția la premise schimbate. Un răspuns corect cu o dovadă invalidă este o categorie separată de un răspuns corect și justificat.

Când un set de date conține ambiguități reale, nu forțăm artificial o singură interpretare drept adevăr absolut. Putem evalua dacă sistemul păstrează interpretările acceptabile și dacă răspunsul lui exprimă corect ce este comun sau divergent între ele.

Compararea cu modele lingvistice și sisteme hibride

Comparațiile trebuie să fixeze informația disponibilă, accesul la instrumente, bugetul și criteriul de succes. Un sistem simbolic care primește interpretarea perfectă nu este comparabil direct cu un model care primește numai text brut. Ambele condiții sunt utile, dar răspund la întrebări diferite.

Testăm un model lingvistic direct, un sistem de circuite cu reguli furnizate, un sistem cu selecție învățată și unul hibrid cu verificare. Menținem verificatorul constant acolo unde vrem să izolăm efectul strategiei de căutare. Raportăm și cazurile în care un model lingvistic rezolvă ceva ce biblioteca simbolică nu poate exprima.

Acuratețea medie trebuie însoțită de acoperire și abținere. Un sistem care răspunde la foarte puține întrebări poate avea precizie excelentă fără competență largă. Un sistem care răspunde la toate poate avea acoperire bună și multe afirmații nesusținute. Curba dintre acoperire și eroare este mai informativă decât o singură cifră.

Fenomene care ar merita urmărite

Un fenomen promițător ar fi creșterea aproape liniară a costului pentru multe ambiguități independente, urmată de o creștere accentuată când aceleași ambiguități devin global corelate. Aceasta ar arăta că partajarea exploatează independența reală, nu doar particularități ale șabloanelor.

Alt fenomen ar fi transferul unui circuit învățat despre posesie la un domeniu cu structură similară, dar vocabular diferit. Un rezultat negativ ar fi nevoia de reantrenare sau de reguli noi pentru fiecare reformulare superficială. În documente reale, putem observa și o limită de cunoaștere: circuitul este corect, dar premisa necesară nu apare nicăieri. Sistemul trebuie să recunoască lipsa, nu să o mascheze prin fluență.

27. Obiecțiile cele mai puternice

„Este doar programare dinamică, cu alt nume"

O parte a mecanismului este, într-adevăr, programare dinamică asupra unei stări semantice potrivite. Nu ar fi onest să revendicăm noutatea acestui principiu. Posibila contribuție constă în reprezentările alese pentru cunoaștere lingvistică, în compunerea garanțiilor și în învățarea interfețelor care fac partajarea posibilă.

Dacă proiectul produce doar o notație nouă pentru metode cunoscute, valoarea poate fi inginerească sau pedagogică, nu o teorie nouă a inteligenței. Această evaluare trebuie acceptată. O integrare foarte bună poate fi utilă fără o revendicare matematică exagerată.

„Tot sensul a fost ascuns în primitive"

Obiecția este validă când operațiile de bază rezolvă deja problemele dificile, iar circuitul doar le conectează nominal. Răspunsul trebuie să fie o analiză a contribuției fiecărei componente. Ce face interpretul lingvistic? Ce face verificatorul? Ce este furnizat manual? Ce este învățat?

Un test de ablație elimină sau înlocuiește o componentă pentru a vedea ce competență dispare. Dacă toată performanța provine dintr-un model extern, proiectul poate rămâne util

ca mediu de audit, dar nu a demonstrat o alternativă de raționament autonom. Un rezultat intermediar nu trebuie împins dincolo de ce arată.

„Interfața suficientă este tot atât de mare ca problema"

În unele clase, acesta poate fi răspunsul corect. O întrebare viitoare arbitrară poate cere orice detaliu al trecutului. Dacă nu restrângem continuările sau nu acceptăm pierderi, compresia poate fi mică sau nulă. Această limită nu trebuie ascunsă prin alegerea unei familii de întrebări convenabile după observarea rezultatelor.

Programul trebuie să identifice clasele în care există structură reutilizabilă și cele în care nu. Poate că sistemele utile vor folosi interfețe locale și temporare, nu o singură esență globală a documentului. O constatare negativă despre comprimarea universală poate conduce la o arhitectură mai bună a memoriei.

„Verificarea semnificației este la fel de grea ca înțelegerea"

Pentru limbaj deschis, obiecția este adesea serioasă. Un verificator nu poate decide exact orice interpretare fără un model suficient al limbajului și al lumii. Separarea propunerii de verificare nu produce gratuit un judecător omniscient.

Totuși, verificări parțiale pot fi valoroase: sursa conține citatul; rolurile unui eveniment sunt consistente; concluzia respectă regulile declarate; nu s-a confundat timpul; un contraexemplu invalidează generalizarea. Trebuie raportată acoperirea verificărilor. O colecție de garanții locale nu trebuie numită adevăr global certificat.

„Probabilitățile reintroduc exact opacitatea pe care voiam să o evităm”

Probabilitățile pot descrie incertitudine legitimă, iar strategiile învățate pot reduce căutarea. Problema nu este prezența lor, ci amestecarea rolurilor. Un scor poate ordona candidați fără să justifice adevărul candidatului. O distribuție poate descrie alternative fără să le transforme în certitudini.

Opacitatea rămâne o problemă de diagnostic și de echitate a selecției. Putem limita rolul componentelor opace, compara euristici și păstra verificarea acolo unde există specificații. Nu rezultă că orice parte a sistemului va deveni intuitiv transparentă pentru orice om.

„Un sistem corect formal poate fi inutil practic”

Da. Poate răspunde numai la întrebări triviale, poate avea costuri mari de modelare sau poate necesita atât de multe clarificări încât utilizatorul renunță. Un program de cercetare trebuie să includă utilitatea sarcinii și costul total al utilizării, nu doar frumusețea garanțiilor.

Domeniile inițiale ar trebui alese acolo unde o concluzie greșită are cost, unde sursele sunt accesibile și unde regulile pot fi precizate: anumite analize de documente, urmărirea cerințelor, verificarea consistenței și raționamentul asupra procedurilor. Aceasta este o alegere de cercetare propusă, nu o promisiune că orice asemenea aplicație este deja rezolvată.

„O explicație elegantă poate deveni un nou tip de iluzie”

Un circuit bine desenat inspiră încredere. Tocmai de aceea trebuie arătate premisele fragile, verificările absente și alternativele nerezolvate. Formalizarea poate produce o iluzie de rigoare când cititorul nu vede câtă interpretare informală precede primul nod.

Răspunsul nu este renunțarea la formalizare, ci extinderea auditului până la granița dintre sursă și model. O mașină de înțelegere ar trebui să facă această graniță mai vizibilă decât este într-un răspuns fluent obișnuit. Dacă o ascunde, a ratat una dintre motivațiile centrale ale proiectului.

28. Etapele unui program deschis și criteriile de continuare

Etapa întâi: vocabular comun și modele verificabile

Primul rezultat necesar este o semantică explicită pentru o clasă mică de circuite: ce sunt valorile, alternativele, relațiile și garanțiile. Se construiesc teste finite în care adevărul de referință este accesibil și se verifică separarea dintre execuție, sinteză și audit.

Criteriul de trecere nu este demonstrarea inteligenței generale. Este absența erorilor de semnificație în clasa exactă declarată, reproducerea independentă a rezultatelor și identificarea costurilor reale. Orice eșec de exactitate trebuie explicat înainte ca arhitectura să fie extinsă peste un fundament instabil.

Etapa a doua: interfețe alese și rafinate automat

Sistemul primește o bibliotecă de abstracții și trebuie să aleagă interfața potrivită unei sarcini. Apoi i se permite să combine abstracții și să le rafineze după contraexemple. Se măsoară costul selecției și al schimbărilor de reprezentare.

Trecerea este justificată dacă adaptarea produce câștiguri pe familii nefolosite pentru reglaj și dacă nu ascunde pierderi semantice. Dacă selecția costă mai mult decât economisește în toate regimurile testate, trebuie revizuită strategia sau restrâns domeniul. Un mecanism adaptiv nu este util doar fiindcă pare mai inteligent.

Etapa a treia: limbaj controlat și memorie revizuibilă

Circuitele sunt conectate la un limbaj suficient de bogat pentru identitate, evenimente, timp, negație și ambiguitate. Se introduce schimbarea întrebării și corectarea surselor. Separăm erorile de interpretare de cele de raționament.

O condiție importantă este posibilitatea de a recupera un răspuns nou din sursa păstrată atunci când interfața anterioară era insuficientă. Alta este retragerea concluziilor după corectarea premiselor. Fără aceste comportamente, sistemul riscă să fie doar un rezolvitor de probleme statice, nu o memorie de cunoaștere evolutivă.

Etapa a patra: învățarea de circuite și reprezentări

Sistemul propune abstracții noi, le compară cu biblioteca și încearcă să le valideze. Cercetătorul nu trebuie eliminat, dar rolul lui trebuie contabilizat. Sunt păstrate atât propunerile acceptate, cât și cele respinse, împreună cu motivele.

Criteriul esențial este transferul: o reprezentare nouă trebuie să ajute probleme care nu au fost folosite pentru inventarea ei. Dacă progresul constă numai în memorarea soluțiilor, numim rezultatul corect și continuăm să căutăm mecanismul de generalizare. Nu redenumim memoria ca înțelegere pentru a salva ipoteza.

Etapa a cincea: documente reale și cercetători independenți

Abia după stabilizarea mecanismelor de bază evaluăm documente reale cu ambiguități, surse incomplete și dezacorduri. Participă cercetători care nu au construit sistemul. Sarcinile sunt alese pentru utilitate și pentru posibilitatea de a evalua erorile, nu doar pentru a produce exemple demonstrative.

Un rezultat matur ar include performanța, costul, acoperirea verificării, cazurile de abținere și costul diagnosticului. Ar arăta unde sistemul este preferabil unei alternative și unde nu. O contribuție care delimitează riguros un domeniu de utilitate poate fi mai valoroasă decât o demonstrație spectaculoasă, dar imposibil de evaluat.

Rolurile cercetătorilor umani și AI

Cercetătorul uman poate aduce intuiții, scopuri, judecăți despre relevanță și interpretări ale eșecurilor. Un partener AI poate propune formalizări, exemple, contraexemple și variante de arhitectură. Niciunul nu trebuie să fie tratat drept verificator infailibil al propriilor propuneri.

O practică sănătoasă separă rolurile de generare și critică, folosește verificatoare independente când este posibil și păstrează deciziile metodologice. O afirmație susținută de mai multe răspunsuri ale aceluiași tip de model nu devine automat confirmare independentă. Independența trebuie să privească mecanismul și sursa dovezii, nu numai numărul de texte produse.

Ce păstrăm din intuiția inițială

Păstrăm ideea că o reprezentare circuitală poate unifica date, cunoaștere și transformări într-un obiect inspectabil. Păstrăm posibilitatea ca reprezentările intermediare să determine dificultatea construirii raționamentului. Păstrăm ambiția de a învăța circuite și interfețe, nu doar de a executa ceea ce a fost scris manual.

Renunțăm la presupunerea că un circuit explicit este automat ușor de găsit, că un fir bine tipizat păstrează toate relațiile sau că o demonstrație formală rezolvă interpretarea lumii. Aceste renunțări nu golesc proiectul. Îi identifică întrebarea centrală.

> Nu căutăm numai un limbaj în care să descriem raționamente. Căutăm reprezentări în care raționamente diferite să își poată împărți viitorul fără să își falsifice sensul.

Dacă această direcție reușește, o parte a progresului nu va apărea ca un circuit din ce în ce mai uriaș. Va apărea ca descoperirea unei frontiere mai mici și mai potrivite: suficient de bogată pentru întrebările care contează, suficient de disciplinată pentru a putea fi continuată și suficient de explicită pentru a putea fi contestată.

Dacă direcția eșuează în anumite clase, un rezultat bine documentat va arăta ce distincții nu pot fi comprimate în forma propusă și unde se mută costul. Acesta este sensul unui program de cercetare deschis: nu protejarea unei intuiții de probe, ci construirea probelor care o pot transforma într-o cunoaștere mai bună.

ANEXE

Atelierul cititorului

Argumente pas cu pas, exerciții cu răspunsuri și un vocabular comun pentru continuarea cercetării.

A. Trei argumente care merită înțelese complet

A.1. De ce o interfață exactă permite reunirea istoriilor

Presupunem două stări s și t cu același rezumat: α(s) = α(t). Presupunem că orice operație admisă F are o versiune F# asupra rezumatelor și că α(F(s)) = F#(α(s)). Mai presupunem că rezultatul relevant R poate fi obținut printr-o funcție h a rezumatului.

Aplicăm aceeași operație F celor două stări. Rezumatele obținute sunt F#(α(s)) și F#(α(t)). Ele sunt egale, fiindcă argumentele sunt egale și operația F# este aceeași. Putem repeta argumentul pentru următoarea operație. După oricâți pași finiți, rezumatele rămân egale. Rezultatele observabile sunt atunci egale, fiindcă ambele se calculează prin h din același rezumat.

Aceasta demonstrează echivalența pentru continuările formate din operațiile declarate. Dacă politica de alegere a operației consultă o parte a stării care nu se află în rezumat, argumentul nu acoperă automat acea politică. Trebuie fie să includem informația consultată, fie să definim continuările ca secvențe externe fixate și să analizăm separat strategia de alegere.

Într-un sistem cu operații parțiale, trebuie păstrată și admisibilitatea: două stări reunite nu pot avea o operație permisă numai într-una, dacă această diferență contează. Într-un sistem nedeterminist, trebuie comparate mulțimile sau distribuțiile de rezultate potrivit semanticii adoptate. Forma simplă de mai sus privește tranziții deterministe și compatibile cu rezumatul.

Ceea ce nu demonstrează argumentul este existența unui rezumat mic, calculabil ieftin sau ușor de învățat. Putem avea proprietatea cu α egală cu identitatea, adică păstrând întrea-

ga stare. Aceasta este exactă și poate să nu comprime nimic. Valoarea cercetării începe când găsim interfețe mult mai mici decât istoriile pe care le reprezintă.

A.2. Cum se păstrează siguranța unei aproximări

Fie C mulțimea stărilor concrete încă posibile și a o descriere abstractă care le include: C este inclusă în γ(a). Presupunem că transformarea abstractă F# este sigură, adică toate rezultatele obținute aplicând F oricărei stări din γ(a) sunt incluse în γ(F#(a)).

Pentru că C este inclusă în γ(a), rezultatele F(C) sunt incluse în F(γ(a)). Iar această ultimă mulțime este inclusă, prin ipoteza siguranței, în γ(F#(a)). Prin urmare, descrierea abstractă nouă include toate rezultatele concrete reale. Argumentul se repetă la fiecare pas.

Acum putem înțelege ce înseamnă o verificare reușită. Dacă toate stările din descrierea abstractă satisfac proprietatea cerută, atunci și toate stările concrete o satisfac. Dacă descrierea abstractă admite un caz rău, nu rezultă neapărat că acel caz este real. Poate fi o alarmă falsă introdusă de aproximare.

Pentru căutarea unei soluții, o intersecție abstractă nevidă cu obiectivul nu este suficientă. Candidatul trebuie realizat concret sau verificat într-o reprezentare mai precisă. În schimb, dacă o supraaproximare sigură a rezultatelor nu intersectează deloc obiectivul, putem respinge ramura, fiindcă nici rezultatele reale nu îl pot atinge.

Acesta este motivul pentru care abstracțiile sunt foarte bune pentru filtrare, dar cer precauție la acceptare. O aceeași reprezentare poate fi sigură pentru a spune „imposibil” și insuficientă pentru a spune „am găsit”, în funcție de direcția logică a testului.

A.3. De ce rezumatele individuale nu păstrează orice relație

Avem relația R formată din perechile (0, 0) și (1, 1). Proiecția ei asupra primei variabile este {0, 1}; proiecția asupra celei de-a doua este tot {0, 1}. Dacă reconstruim posibilitățile doar din aceste două proiecții, obținem produsul lor: patru perechi.

Două dintre perechi, (0, 1) și (1, 0), nu aparțineau relației inițiale. Operația „păstrează fiecare componentă separat” a pierdut dependența de egalitate. Nu a pierdut nicio valoare individual posibilă. Tocmai de aceea o verificare care privește numai domeniile fiecărui fir poate rata problema.

Reconstrucția prin produs este o supraaproximare sigură a lui R. Ea poate fi folosită pentru anumite teste, dar nu ca și cum ar fi relația exactă. Dacă o continuare întreabă dacă x = y, R răspunde întotdeauna afirmativ, în timp ce produsul admite și răspunsul negativ.

O reparație compactă este să păstrăm cele două domenii și constrângerea x = y. Nu a fost necesară enumerarea tuturor perechilor. În alte relații, o reparație la fel de mică poate să nu existe în limbajul ales. De aceea, cercetarea interfețelor este și cercetarea unui vocabular al relațiilor utile.

B. Exerciții conceptuale și răspunsuri comentate

1. Un rezumat pentru medie

O mașină citește numere pe rând. Păstrează numai suma lor și trebuie să răspundă la final care este media. Este interfața suficientă?

Nu. Listele [2, 4] și [6] au aceeași sumă, dar medii diferite. Numărul elementelor este distincția lipsă. Perechea „sumă, număr” poate fi actualizată compozițional la adăugarea unui element și permite calcularea mediei pentru liste nevide. Cazul listei vide trebuie tratat explicit, nu ascuns într-o împărțire nepermisă.

2. Aceeași medie, altă întrebare

Presupunem că păstrăm suma și numărul. Putem decide dacă toate elementele listei au fost egale?

Nu în general. Listele [1, 3] și [2, 2] au aceeași sumă și același număr de elemente. Putem adăuga o reprezentare potrivită, de exemplu o valoare de referință și un indicator al egalității tuturor elementelor cu ea. Exercițiul arată că o interfață suficientă nu este o esență universală a datelor.

3. Tipul corect, sensul greșit

O operație primește două numere și le adună. Primul este un preț în euro, al doilea o temperatură în grade Celsius. Sistemul verifică numai tipul numeric. Ce lipsește?

Lipsește tipul semantic sau contractul care precizează ce mărimi se pot combina. Verificarea executabilității nu dovedește relevanța operației. Soluția nu este interzicerea aritmeticii generale, ci distingerea nivelului brut de date de folosirea justificată într-un model.

4. O intersecție nevidă

Analiza spune că o operație poate produce orice număr între 0 și 10. Obiectivul este 7. Putem declara că există o execuție care produce 7?

Nu, dacă intervalul este doar o supraaproximare. Rezultatele reale ar putea fi numai 0 și 10. Intersecția arată compatibilitate abstractă, nu realizabilitate. Trebuie verificat un martor concret sau rafinată reprezentarea până când existența devine justificată.

5. O contradicție din surse

Un document afirmă că Ana este la laborator la ora 10. Altul afirmă că nu este la laborator la aceeași oră. Ce ar trebui să producă sistemul?

Mai întâi trebuie să stabilească dacă se referă la aceeași persoană, același loc și aceeași semnificație a momentului. Dacă da și dacă ambele afirmații sunt acceptate ca premise despre aceeași lume, trebuie semnalat conflictul. Poate păstra separat faptul că fiecare sursă a făcut afirmația. Nu trebuie să aleagă automat sursa preferată și nici să transforme incompatibilitatea în permisiunea de a răspunde orice.

6. O regulă care uită o precondiție

Un circuit rescrie x/x ca 1. Ce trebuie verificat înainte de a introduce regula într-o clasă de egalitate?

Trebuie precizată semantica numerelor și domeniul de definiție. În aritmetica obișnuită, condiția x diferit de zero este esențială. Într-o semantică numerică particulară pot exista și alte cazuri speciale. O regulă corectă sub o precondiție nu poate fi aplicată necondiționat doar fiindcă simplifică expresia.

7. O ambiguitate irelevantă

„Ana îi dă cheia Mariei. Ea pleacă.” Întrebarea este cine are cheia imediat după transfer. Trebuie aleasă identitatea pronumelui?

Nu, în modelul restrâns în care transferul este actual și încheiat, iar plecarea nu schimbă posesia la momentul întrebat. Ambele interpretări dau Maria. Dacă întrebarea devine cine pleacă, ambiguitatea trebuie păstrată sau rezolvată prin informație suplimentară. Relevanța este relativă la continuare.

8. O explicație cu două surse dependente

Două articole susțin aceeași afirmație, dar ambele copiază un singur raport. Putem considera că avem două confirmări independente?

Nu doar din numărul articolelor. Proveniența arată dependența comună. Putem avea două reproduceri ale aceleiași surse, nu două observații independente. Un sistem poate număra derivările, sursele documentare și originile informației, dar acestea sunt mărimi diferite.

9. Un rezultat negativ ambiguu

Expanderul nu găsește niciun circuit înainte de terminarea bugetului. Ce poate afirma?

Poate afirma că nu a găsit o soluție în spațiul și bugetul explorate. Nu poate declara imposibilitatea fără o justificare suplimentară. Dacă biblioteca nu conține operațiile necesare, lipsa soluției spune ceva despre limbajul de căutare, nu despre imposibilitatea sarcinii în general.

10. O comprimare care șterge proveniența

Două circuite produc aceeași concluzie, unul dintr-o sursă acceptată, altul dintr-o ipoteză neverificată. Putem păstra numai unul, ales arbitrar?

Numai dacă proveniența nu influențează nicio continuare relevantă și dacă rezultatul cerut nu impune un anumit suport. În aplicații epistemice, această condiție este adesea

falsă. Putem reuni valoarea și păstra justificările distincte sau putem include statutul sursei în interfață. Echivalența valorică nu este automat echivalență epistemică.

11. O reprezentare vectorială exactă pe probe

Un vector permite recuperarea corectă a tuturor rolurilor dintr-un set de o sută de exemple. Ce concluzie este justificată?

Că reprezentarea și procedura de recuperare au reușit pe acele exemple, în condițiile testate. Nu rezultă exactitatea universală și nici stabilitatea la creșterea numărului de relații. Următoarele teste ar trebui să varieze suprapunerea, similaritatea obiectelor, rolurile și dimensiunea memoriei, incluzând cazuri adversariale.

12. O abstracție care crește fără oprire

Fiecare contraexemplu determină sistemul să adauge un caz special. După o mie de exemple, reprezentarea conține aproape o mie de excepții. Este aceasta învățarea unei interfețe suficiente utile?

Poate fi o memorie exactă a cazurilor, dar nu demonstrează o comprimare structurală utilă. Trebuie comparată cu memorarea directă și evaluată pe cazuri noi. Un rezultat negativ poate sugera că limbajul abstracțiilor este nepotrivit sau că familia de probleme nu are regularitatea căutată. Nu trebuie ascuns prin redenumirea excepțiilor ca „concepte”.

13. Predicție sau intervenție

Un sistem prezice că oamenii cu umbrele ude au fost în ploaie. Poate deduce că udarea unei umbrele va produce ploaie?

Nu. Prima relație este predictivă, în modelul observațiilor. A doua privește o intervenție asupra lumii. Este necesar un model al mecanismelor cauzale. Graful prin care sistemul calculează predicția nu este automat graful cauzal al fenomenului.

14. Un câștig care dispare după contabilizare

Un sistem răspunde de zece ori mai repede după o compilare costisitoare. Cum decidem dacă este mai eficient?

Precizăm numărul interogărilor și frecvența actualizărilor. Adunăm compilarea, verificarea, memoria și costul răspunsurilor. Putem obține un prag de reutilizare de la care metoda devine avantajoasă. Fără acest regim de utilizare, comparația „mai rapid” este incompletă.

15. Ce ar merita numit progres în înțelegere

Un sistem răspunde corect unei întrebări, apoi își schimbă răspunsul după o reformulare care nu schimbă sensul. Alt sistem se abține la prima întrebare, explică ambiguitatea și răspunde după o clarificare relevantă. Care rezultat este mai informativ?

Depinde de sarcină, dar al doilea oferă o dovadă mai clară a controlului asupra informației lipsă. Primul succes izolat poate fi accidental. Evaluarea înțelegerii trebuie să urmărească familii de variații și reacția la ele, nu doar un răspuns norocos. Capacitatea de a localiza insuficiența reprezentării face parte din competența urmărită.

C. Glosar de lucru

Abstracție. Reprezentare care păstrează anumite proprietăți ale unei stări și omite altele. Valoarea ei depinde de operațiile și întrebările urmărite.

Abținere. Decizie explicită de a nu emite o concluzie nejustificată. Trebuie deosebită de eșecul tehnic și de imposibilitatea demonstrată.

Admisibilitate. Condiția în care o operație poate fi aplicată legitim, nu doar executată mecanic.

Algebră relațională. Familie de operații asupra relațiilor, precum selectarea și combinarea tabelelor. În sens mai larg, cartea folosește algebra relațiilor pentru reguli de combinare care păstrează dependențe.

Alternativă. Caz posibil care nu trebuie combinat arbitrar cu consecințele unui caz incompatibil.

Auditabilitate. Posibilitatea de a examina premisele, transformările și limitele unui rezultat. Auditabilitatea practică include costul acestui examen.

Bază de încredere. Componentele și presupunerile de care depinde validitatea verificării întregului sistem.

Bisimulare. Relație între stări care își pot corespunde reciproc pașii observabili într-un model de tranziții.

CEGAR. Rafinarea unui model abstract după constatarea că un contraexemplu aparent nu este realizabil concret.

CEGIS. Îmbunătățirea unui candidat la program folosind intrări pe care acesta încalcă specificația.

Certificat. Obiect care permite verificarea unei afirmații într-un model declarat. Nu certifică automat adevărul extern al premiselor.

Circuit. Structură de operații și dependențe între valori. În carte, termenul nu impune componente electronice sau valori binare.

Compilarea cunoașterii. Transformarea unei reprezentări pentru a face anumite interogări sau operații mai eficiente, cu un cost anterior explicit.

Compoziționalitate. Posibilitatea de a deduce comportamentul întregului din componente și din regulile conectării lor.

Congruență. Echivalență păstrată de operațiile și contextele relevante. Permite înlocuirea unei componente fără schimbarea observației urmărite.

Conținut materializat. Valoarea produsă prin execuție, de exemplu un tabel de evenimente; nu este aceeași cu sursa circuitului.

Contract semantic. Precizarea relației dintre intrarea și ieșirea unei operații, inclusiv condițiile necesare.

Contraexemplu. Caz concret care încalcă o afirmație generală sau distinge două stări reunite nejustificat.

Corelație. Dependență între valori care nu poate fi reconstruită întotdeauna din descrierile fiecărei valori separat.

Domeniu abstract. Spațiul descrierilor folosite de o analiză, împreună cu operațiile și ordinea relevante.

E-graph. Structură partajată pentru expresii reunite prin egalități acceptate și prin compatibilitatea lor cu contextul.

Echivalență contextuală. Imposibilitatea de a distinge două construcții prin continuările și observațiile admise.

Epocă. Un interval conceptual de calcul cu anumite versiuni și ipoteze fixate; schimbările pot deschide o epocă nouă.

Expander. Componenta propusă care dezvoltă și selectează continuări de circuit în raport cu un obiectiv și cu informația disponibilă.

Frontieră. Ansamblul valorilor și relațiilor prin care o construcție parțială poate interacționa cu viitoarele operații.

Garanție exactă. Păstrarea proprietății declarate fără pierderi, în domeniul și sub premisele precizate.

Interfață suficientă. Rezumat care păstrează tot ce cer continuările și observațiile relevante, în clasa analizată.

Interpretare abstractă. Calcul sistematic cu descrieri ale stărilor, de regulă pentru aproximări sigure ale comportamentului.

Latice. Structură ordonată în care descrierile pot fi combinate prin operații corespunzătoare unor limite comune.

Martor. Exemplu concret care demonstrează existența unei soluții, precum un circuit și execuția lui verificabilă.

Monotonie. Păstrarea unei ordini când aplicăm o operație. În propagarea informației, permite raționarea despre rafinare stabilă.

Precondiție. Cerință asupra intrării sau contextului care face aplicabilă o operație ori garantează o proprietate a ieșirii.

Proveniență. Structura surselor și transformărilor de care depinde o valoare sau o concluzie.

Punct fix. Stare în care reaplicarea operației sau a regulilor considerate nu mai schimbă descrierea.

Rafinare. Introducerea unei distincții sau restrângerea posibilităților. Trebuie deosebită de revizuirea unei premise retrase.

Semiring. Structură cu două operații folosite pentru alternative și compunere, sub legi algebraice potrivite calculului urmărit.

SSA. Formă de reprezentare cu un singur loc de atribuire pentru fiecare nume static de rezultat; nu exclude buclele programului.

Supraaproximare. Descriere care include toate posibilitățile reale și poate include posibilități suplimentare.

Treewidth. Măsură a lățimii structurale a unui graf, relevantă pentru anumite descompuneri și algoritmi; nu descrie singură toată dificultatea.

Vedere specifică întrebării. Reprezentare temporară optimizată pentru un obiectiv, derivată dintr-o memorie sau sursă mai bogată.

VSA/HDC. Familii de reprezentări distribuite de dimensiune mare și operații pentru codificare, compunere și recuperare.

Widening. Tehnică de accelerare a stabilizării unei analize printr-o aproximare mai largă, cu un compromis de precizie.

D. Fișa unui rezultat publicabil

Afirmația și domeniul

Orice rezultat al acestui program ar trebui să înceapă cu o afirmație delimitată: ce familie de circuite, ce operații, ce reprezentare și ce întrebări sunt acoperite. Formulări precum „înțelege textul” sau „elimină complexitatea” trebuie înlocuite cu competențe și condiții observabile.

Ce a fost furnizat și ce a fost învățat

Se precizează regulile scrise de cercetător, schemele de date, componentele externe, interfețele predefinite și contribuțiile învățate. Costul construirii bibliotecii și al verificatoarelor nu dispare din explicație. Cercetătorul viitor trebuie să poată identifica locul exact al contribuției.

Dovada și verificarea

Se separă demonstrațiile matematice, verificările exhaustive finite, testele empirice și observațiile exploratorii. Pentru rezultate numerice se arată ce a fost măsurat și ce a fost cal-

culat prin formulă. O verificare independentă trebuie să reducă riscul repetării aceleiași erori, nu doar să ruleze încă o dată aceeași procedură.

Costul complet

Se raportează învățarea, compilarea, căutarea, verificarea, memoria și actualizarea. Numărul extinderilor evaluate este util, dar nu substituie timpul atunci când extinderile au costuri foarte diferite. Dimensiunea intrării trebuie exprimată și în biți când valorile numerice cresc.

Cazurile care nu au funcționat

Se publică eșecurile, inconsecvențele, cazurile nedecise și depășirile de buget. Se arată dacă problema este în interpretare, reprezentare, selecție, verificare sau exprimare. Un rezultat negativ bine localizat este mai util decât o promisiune generală fără diagnostic.

Ce ar schimba concluzia

Raportul se încheie cu probele care ar invalida pretenția și cu experimentele necesare pentru extinderea ei. Această secțiune nu este o formalitate. Ea oferă cercetătorilor umani și AI o modalitate concretă de a continua fără a moșteni necritic entuziasmul autorilor.

Bibliografie adnotată

Referințe primare și documentație oficială

Titlurile sunt legături către publicații sau pagini oficiale. Notele indică rolul fiecărei surse în programul de cercetare. Anul publicării este separat de cel al preprintului atunci când această diferență este relevantă. Data consultării surselor online: 11 septembrie 2026.

[1] Cytron, R.; Ferrante, J.; Rosen, B. K.; Wegman, M. N.; Zadeck, F. K. (1991). Efficiently Computing Static Single Assignment Form and the Control Dependence Graph. ACM Transactions on Programming Languages and Systems.

Fundamentul istoric pentru SSA în compilatoare. Relevant pentru numele stabile ale rezultatelor și dependențele de control; nu oferă singur o semantică a înțelegerii.

[2] LLVM Project (documentație online). LLVM Language Reference Manual. Documentație oficială; consultată la 11 septembrie 2026.

Referință tehnică pentru reprezentarea SSA, control, tipuri și operații. Ajută la separarea convenției de atribuire de presupuneri suplimentare despre puritate, memorie sau bucle.

[3] Fong, B.; Spivak, D. I. (2018). Seven Sketches in Compositionality: An Invitation to Applied Category Theory. arXiv:1803.05316; volum publicat ulterior, în 2019.

Introducere în matematica sistemelor compozabile. Oferă vocabular pentru relații, procese și traduceri care păstrează structură. În carte este folosită ideea compunerii, nu presupusă familiaritatea cu formalismul.

[4] Coecke, B. (2019; revizie 2020). The Mathematics of Text Structure. arXiv:1904.03478.

Precedent direct pentru textul reprezentat circuital în DisCoCirc. Important pentru atribuirea corectă a ideilor și pentru studierea actualizării compoziționale a semnificațiilor.

[5] Cousot, P.; Cousot, R. (1977). Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints. POPL 1977, pp. 238–252.

Lucrarea fondatoare a interpretării abstracte. Reper pentru relația dintre stările concrete, descrierile aproximative și calculele prin puncte fixe; garanția principală nu trebuie confundată cu exactitatea.

[6] Radul, A.; Sussman, G. J. (raport tehnic online). Revised Report on the Propagator Model. MIT CSAIL; consultat la 11 septembrie 2026.

Model de calcul cu celule care acumulează informație parțială și componente care propagă consecințe. Foarte apropiat conceptual de firele care transportă cunoaștere rafinabilă.

[7] Goldreich, O. (2001). Foundations of Cryptography, Volume I: Basic Tools. Cambridge University Press; pagina autorului.

Fundament pentru distincția dintre calculul direct, inversare și verificare. Funcțiile unidirecționale ilustrează de ce cunoașterea algoritmului nu oferă automat o căutare eficientă.

[8] Vaandrager, F.; Midya, A. (2020). A Myhill-Nerode Theorem for Register Automata and Symbolic Trace Languages. ICTAC 2020; arXiv:2007.03540, versiune extinsă disponibilă.

Reper pentru reunirea istoriilor în funcție de ceea ce continuările pot distinge. Extensia către registre subliniază că relațiile dintre valori pot face parte din starea relevantă.

[9] Shalizi, C. R.; Crutchfield, J. P. (1999; publicare 2001). Computational Mechanics: Pattern and Prediction, Structure and Simplicity. arXiv:cond-mat/9907176.

Studiază reprezentări ale proceselor prin echivalența predicțiilor viitoare. Utilă pentru legătura dintre istorie, stare și suficiență predictivă; nu trebuie confundată cu identificarea cauzală intervențională.

[10] Darwiche, A.; Marquis, P. (2002). A Knowledge Compilation Map. Journal of Artificial Intelligence Research; copie arXiv:1106.1819.

Compară limbaje de reprezentare după compactitate, interogări și transformări eficiente. Reper pentru alegerea reprezentării după utilizare și pentru contabilizarea costului compilării.

[11] Mateescu, R.; Dechter, R.; Marinescu, R. (2008). AND/OR Multi-Valued Decision Diagrams (AOMDDs) for Graphical Models. JAIR 33, pp. 465–519; copie arXiv:1401.3448.

Explică exploatarea independențelor și a descompunerilor grafice. Relevant pentru frontiere, separatoare și diferența dintre numărul istoriilor și dimensiunea unei reprezentări partajate.

[12] Darwiche, A. (2020). An Advance on Variable Elimination with Applications to Tensor-Based Computation. arXiv:2002.09320.

Arată importanța dependențelor funcționale pentru eliminarea variabilelor. Susține cercetarea structurii algebrice, nu doar a conectivității grafului, ca sursă de eficiență.

[13] Polikarpova, N.; Kuraj, I.; Solar-Lezama, A. (2016). Program Synthesis from Polymorphic Refinement Types. PLDI 2016; preprint arXiv:1510.08419.

Precedent pentru sintetizarea programelor cu ajutorul unor tipuri care conțin condiții semantice. Relevant pentru transformarea obiectivului în cerințe asupra componentelor.

[14] Guo, Z.; James, M.; Justo, D.; Zhou, J.; Wang, Z.; Jhala, R.; Polikarpova, N. (2020). Program Synthesis by Type-Guided Abstraction Refinement. POPL 2020; arXiv:1911.04091.

Prezintă TYGAR și folosirea rafinării abstracțiilor în sinteză. Important pentru ideea de a începe cu o descriere mai grosieră și de a introduce distincții când sunt necesare.

[15] Alur, R. și colaboratorii (2013). Syntax-Guided Synthesis. FMCAD 2013; manuscris disponibil pe pagina autorului.

Separă spațiul sintactic al programelor admise de specificația comportamentului. Oferă cadrul pentru a discuta explicit ce caută un expander și ce verifică evaluatorul candidatului.

[16] Clarke, E.; Grumberg, O.; Jha, S.; Lu, Y.; Veith, H. (2000). Counterexample-Guided Abstraction Refinement. CAV 2000, LNCS 1855, pp. 154–169.

Lucrarea de referință pentru CEGAR. Un contraexemplu fals poate indica o distincție pierdută în modelul abstract, nu o eroare a sistemului concret.

[17] Willsey, M.; Nandi, C.; Wang, Y. R.; Flatt, O.; Tatlock, Z.; Panchekha, P. (2021). egg: Fast and Extensible Equality Saturation. POPL 2021; arXiv:2004.03082.

Reper pentru reprezentarea partajată a expresiilor echivalente. Trebuie deosebită egalitatea validată de simpla păstrare a interpretărilor alternative.

[18] Goodman, J. (1999). Semiring Parsing. Computational Linguistics 25(4), pp. 573–606.

Arată cum o structură de analiză poate fi evaluată cu algebre diferite. Utilă pentru a distinge existența, numărarea, costul și ponderarea derivărilor.

[19] Grädel, E.; Tannen, V. (2024). Provenance Analysis and Semiring Semantics for First-Order Logic. arXiv:2412.07986.

Cadru pentru urmărirea dependențelor logice prin semantica provenienței. Relevant pentru justificări alternative și pentru precauțiile necesare când intră în joc negația.

[20] Soufflé Project (documentație online). Soufflé Tutorial. Documentație oficială; consultată la 11 septembrie 2026.

Introducere practică în relații și reguli Datalog. Este un punct de intrare accesibil pentru intuiția derivării de fapte, nu un substitut al semanticii limbajului natural.

[21] Vaswani, A. și colaboratorii (2017). Attention Is All You Need. NeurIPS 2017; arXiv:1706.03762.

Reper pentru arhitectura Transformer. Este citată aici pentru faptul că reprezentări distribuite și operații neuronale pot fi descrise prin grafuri explicite de calcul.

[22] Yeung, C.; Zou, Z.; Jeong, S.; Huang, W.; Bastian, N. D.; Imani, M. (2024; versiune revizuită în 2026). Generalized Holographic Reduced Representations. arXiv:2405.09689.

Cercetare asupra compoziționalității în reprezentări distribuite și a operațiilor de legare. Oferă o direcție de comparație, nu dovada că stocarea holografică elimină pierderea informației.

[23] Ellis, K. și colaboratorii (2020). DreamCoder: Growing Generalizable, Interpretable Knowledge with Wake-Sleep Bayesian Program Learning. arXiv:2006.08381.

Precedent pentru biblioteci simbolice învățate și strategii neuronale de explorare. Relevant pentru separarea reprezentării programelor de mecanismul care le caută.

[24] Tishby, N.; Pereira, F. C.; Bialek, W. (1999; preprint 2000). The Information Bottleneck Method. arXiv:physics/0004057.

Formalizează comprimarea care păstrează informația relevantă unei ținte. Ajută la discutarea compromisului dintre memorie și predicție, fără garanții universale despre toate întrebările viitoare.

[25] Strouse, D. J.; Schwab, D. J. (2016). The Deterministic Information Bottleneck. arXiv:1604.00268.

Explorează o formulare alternativă a comprimării informaționale. Termenul determinist nu trebuie confundat cu eliminarea incertitudinii din lume sau cu o dovadă de echivalență semantică.

[26] Grünwald, P. (2004). A Tutorial Introduction to the Minimum Description Length Principle. arXiv:math/0406077.

Fundament pentru evaluarea împreună a complexității modelului și a datelor rămase de descris. Relevant pentru costul bibliotecilor și al excepțiilor, nu pentru identificarea automată a adevărului.

[27] Pearl, J. (2009). Causal Inference in Statistics: An Overview. Statistics Surveys 3, pp. 96–146; raport disponibil pe pagina autorului.

Clarifică rolul modelelor cauzale și diferența dintre observație și intervenție. Important pentru a nu confunda dependențele unui calcul cu mecanismele cauzale ale lumii.

[28] Tafjord, O.; Dalvi Mishra, B.; Clark, P. (2021). ProofWriter: Generating Implications, Proofs, and Abductive Statements over Natural Language. Preprint arXiv:2012.13048, depus în 2020.

Reper pentru evaluarea concluziilor împreună cu dovezile în sarcini lingvistice delimitate. Nu trebuie extrapolat la întreaga competență de înțelegere a documentelor reale.

[29] Saha, S.; Ghosh, S.; Srivastava, S.; Bansal, M. (2020). PRover: Proof Generation for Interpretable Reasoning over Rules. arXiv:2010.02830.

Lucrare despre generarea de dovezi pentru inferențe peste reguli. Relevantă pentru măsurarea separată a răspunsului corect și a justificării valide.

[30] Koehler, T.; Trinder, P.; Steuwer, M. (2021). Sketch-Guided Equality Saturation: Scaling Equality Saturation to Complex Optimizations of Functional Programs. arXiv:2111.13040.

Studiază ghidarea unui spațiu de rescrieri partajate. Este un avertisment util că folosirea unui e-graph nu înlătură automat problema dimensiunii căutării.

[31] Nandi, C. și colaboratorii (2021). Rewrite Rule Inference Using Equality Saturation. arXiv:2108.10436.

Precedent pentru descoperirea unor reguli de egalitate folositoare. Relevant pentru învățarea circuitelor și a transformărilor, cu condiția verificării semanticii regulilor propuse.

[32] Dannert, K. M.; Grädel, E.; Naaf, M.; Tannen, V. (2019). Generalized Absorptive Polynomials and Provenance Semantics for Fixed-Point Logic. arXiv:1910.07910.

Extinde analiza provenienței către logici cu puncte fixe. Arată de ce alegerea algebrei justificărilor trebuie adaptată recursiei și proprietăților logice urmărite.